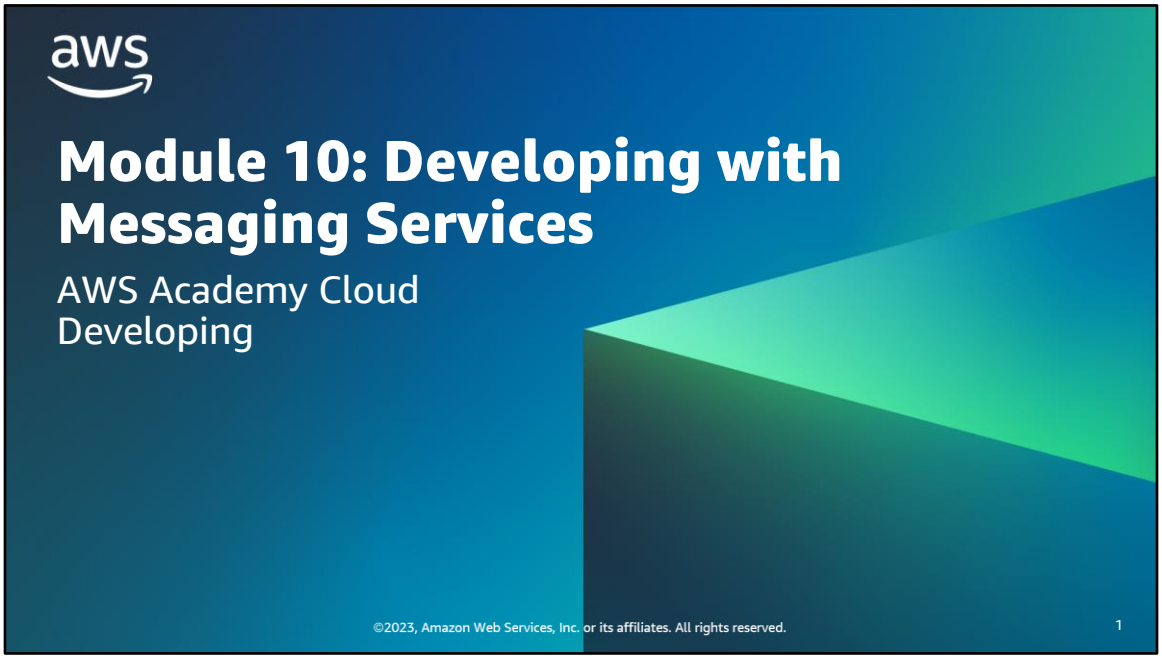
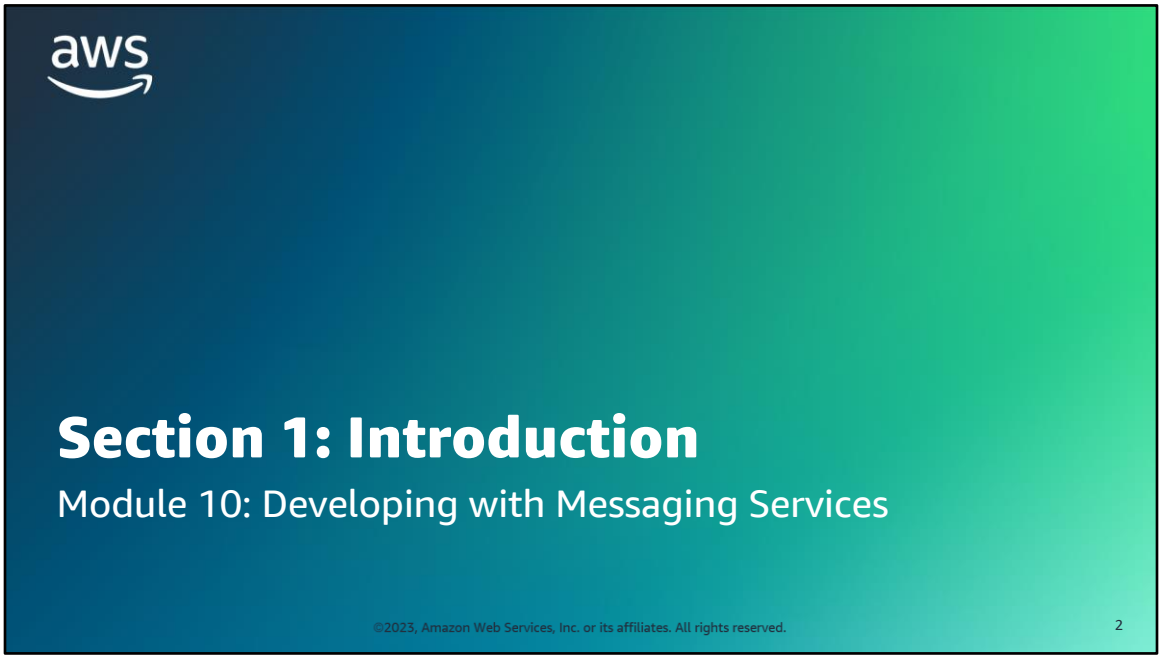
AWS Academy Cloud Developing
Module 10 Student Guide
Version 2.0.6

200-ACCDEV-20-EN-SG

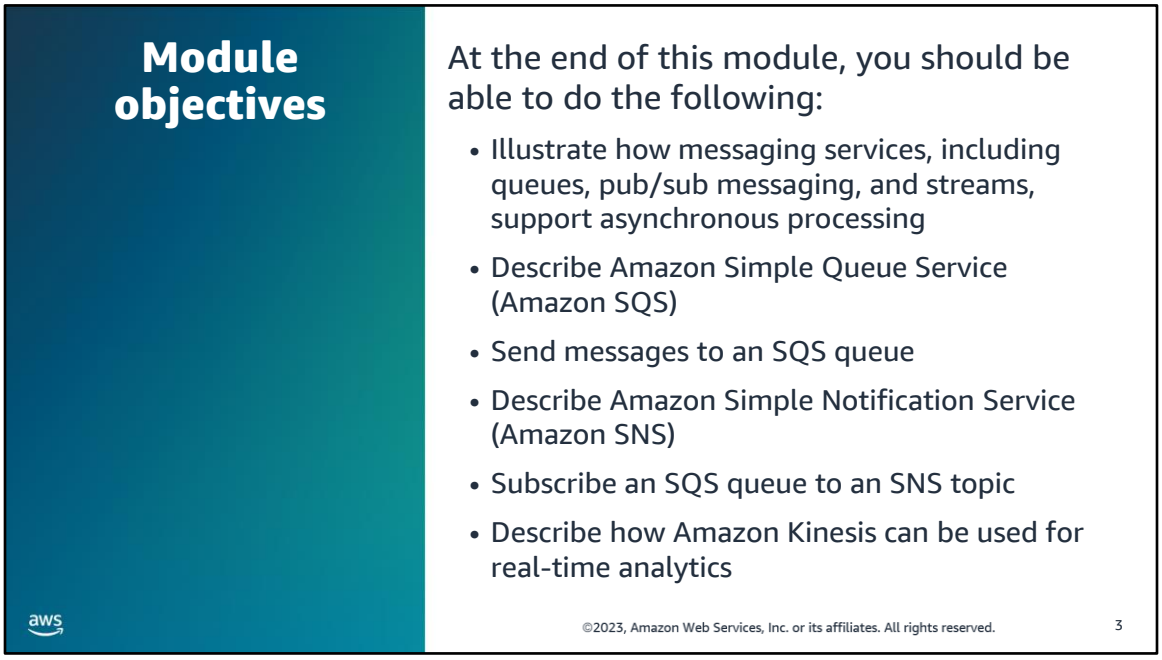
Contents



Welcome to Module 10: Developing with Messaging Services.

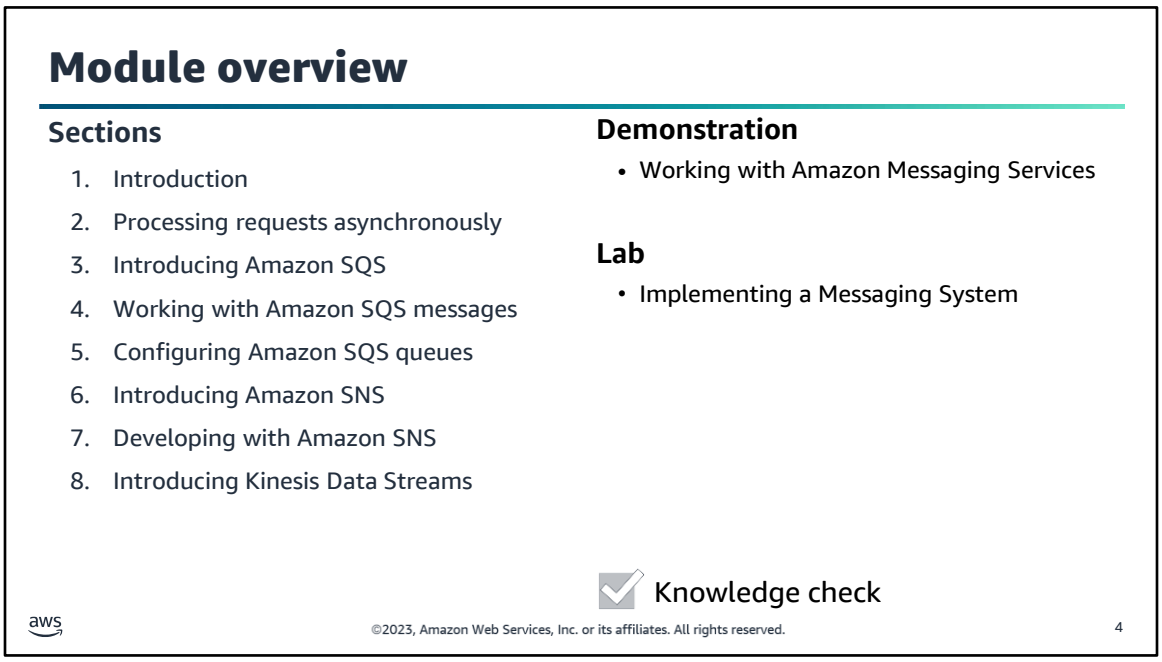


Section 1: Introduction.



At the end of this module, you should be able to do the following:

- Illustrate how messaging services, including queues, pub/sub messaging, and streams, support asynchronous processing
- Describe Amazon Simple Queue Service (Amazon SQS)
- Send messages to an SQS queue
- Describe Amazon Simple Notification Service (Amazon SNS)
- Subscribe an SQS queue to an SNS topic
- Describe how Amazon Kinesis can be used for real-time analytics



This module includes the following sections:

1. Introduction
2. Processing requests asynchronously
3. Introducing Amazon SQS
4. Working with Amazon SQS messages
5. Configuring Amazon SQS queues
6. Introducing Amazon SNS
7. Developing with Amazon SNS
8. Introducing Kinesis Data Streams

This module also includes:

- A demonstration covering Amazon SQS and Amazon SNS
- A lab where you work with Amazon SQS and Amazon SNS

Finally, you will complete a knowledge check to test your understanding of key concepts covered in this module.

Café business requirement

Currently, updating the coffee inventory on the website is a manual process. Sofía would like to find an easier way to keep the inventory updated. She considers setting up a messaging system to automatically receive and process inventory updates.

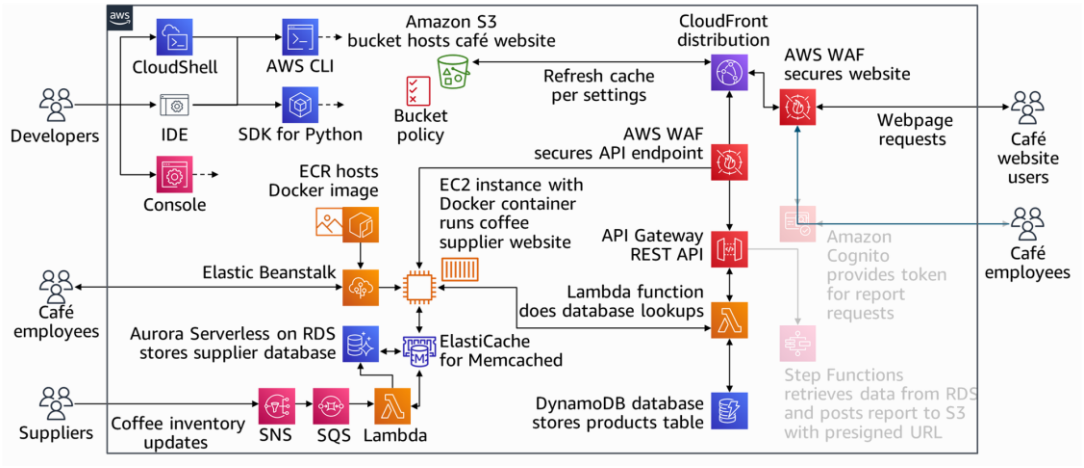
aws

5

Currently, updating the coffee inventory on the website is a manual process. Sofía would like to find an easier way to keep the inventory updated. She considers setting up a messaging system to automatically receive and process inventory updates.

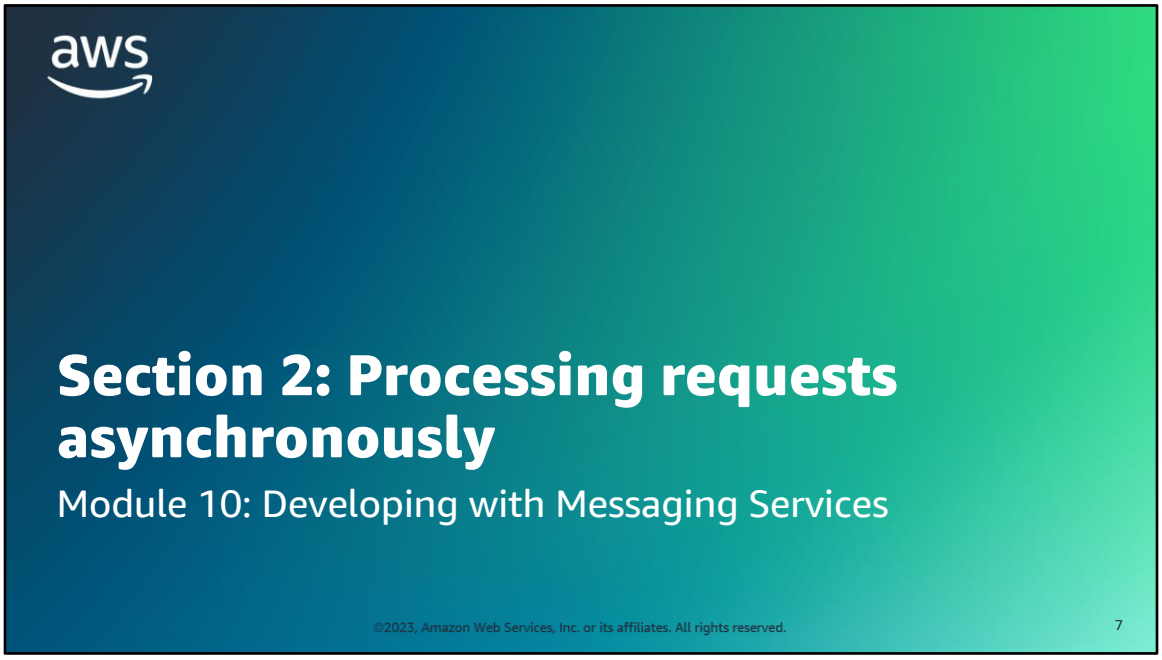
Mateo, a café regular and AWS consultant, suggested using the Amazon Simple Notification Service (Amazon SNS) to receive messages from the suppliers and an Amazon Simple Queue Service (Amazon SQS) queue to store the messages until they are processed.

Messaging services as part of developing a cloud application



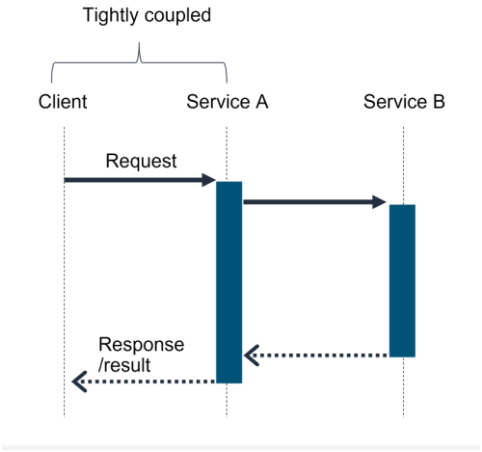
The diagram on this slide gives an overview of the application that you will build through the labs in this course. The highlighted portions are relevant to this module.

As highlighted in the diagram, you will use Amazon SQS and Amazon SNS to set up a system to receive, queue, and send the coffee inventory information.



Section 2: Processing requests asynchronously.

Synchronous processing



aws

8

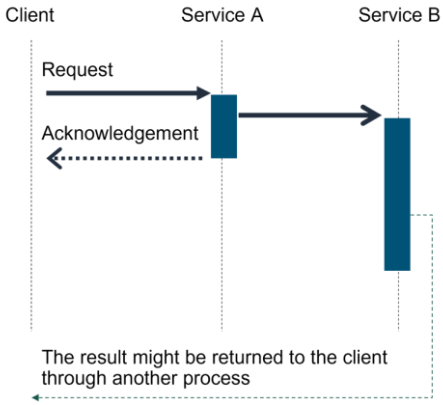
In a *synchronous* model, the client (producer) generates a request message to the consumer. Then, the client waits until the message is processed and the client gets a response from the consumer. This is a relatively simple interaction common to create, read, update, and delete (CRUD) API actions.

In the example of the coffee shop, this is similar to ordering a coffee at the counter and getting the cup of coffee immediately before you walk away. This action is complete before the next person orders.

In the example diagrammed on the slide, the client makes a request to service A. Service A then calls service B. Service A waits for service B to complete its actions and respond to service A. That response then goes back to the client.

In this model, strong interdependency exists between the producer and the consumer. This interdependency creates a tightly coupled system. The disadvantage of a tightly coupled system is that *it is not fault tolerant*. This means that if any component of the system fails, then the entire system will fail. If the consumer fails while processing a message, then the producer will be forced to wait until that message gets processed (assuming that the message is not lost). Additionally, if new consumer instances are launched to recover from a failure or to keep up with an increased workload, the producer must be explicitly made aware of the new consumer instances. In this scenario, the producer is tightly coupled with the consumer, and the coupling is prone to brittleness.

Asynchronous processing



aws

9

In contrast to a synchronous model is *asynchronous* processing. In this design, the client (producer) sends a request and might get an acknowledgement that the event was received. However, the client doesn't receive a response that includes the results of the request.

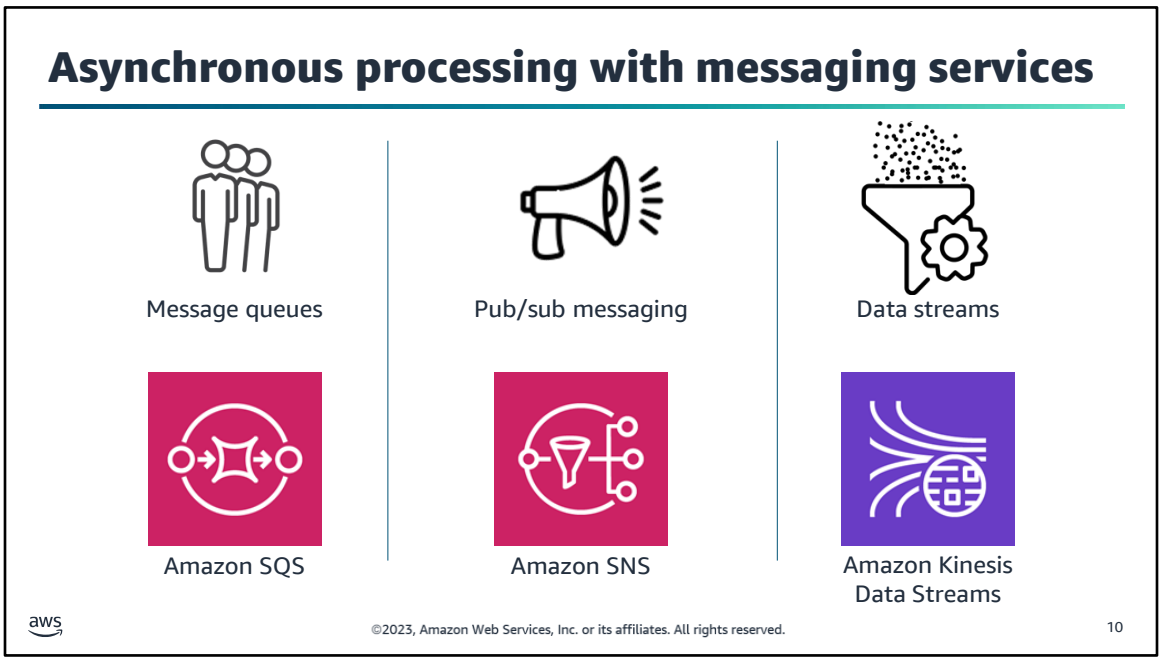
In the coffee shop example, this is similar to placing an order that is delivered later. You get your order number, and then the next person in line can order without waiting for your order to be fulfilled. Someone brings you the order when it is ready. A problem with completing your order does not prevent the person behind you in line from ordering.

In the example diagrammed on the slide, when the client makes a request to service A, service A surfaces the event to its consumers (in this example, service B) and moves on.

The disadvantage is that service B does not have a channel to pass back results to service A. You need to write your applications to get the results and deal with errors asynchronously. Although you might be used to writing code that has explicit coupling, the coupling is not actually needed for the type of processing you are doing.

The advantage of an asynchronous approach is that you reduce the dependencies on downstream activities, and this reduction improves responsiveness back to the client. This means that you don't need to put logic into your code to deal with long wait times or handle errors that might occur downstream. After your client has successfully handed off the request, you can move on.

The producer does not need to wait for the first message to be read. Instead, the producer can continue to generate messages whether the consumer is processing them or not. The producer is not impacted if the consumer fails to process the message. This means much less interdependency between the producer and the consumer, which makes the system more loosely coupled with greater fault tolerance.

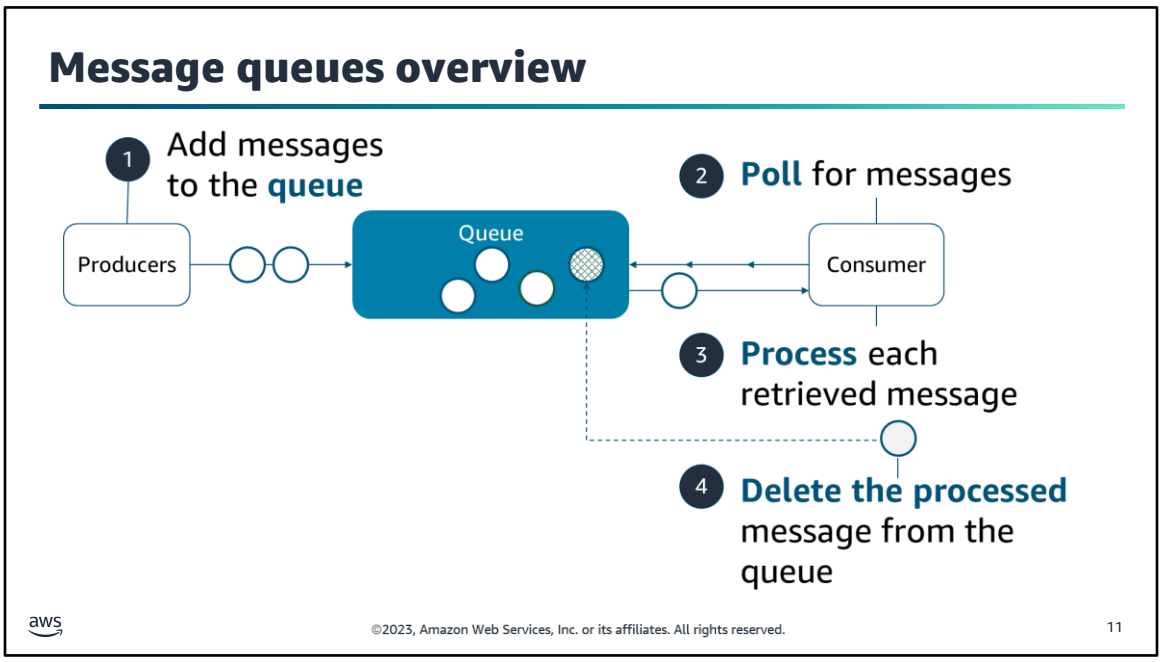


In this module, you'll learn about three types of messaging services that you can use for asynchronous processing:

- Message queues
- Publish/subscribe (pub/sub) messaging
- Data streams

The type of messaging service that you choose depends on your use case. The next few slides provide an overview of these three types of messaging services. In each type, producers send requests (messages) to a service and do not wait for a result. The messaging service provides an interim location from which consumers get messages. This provides an asynchronous, decoupled design between your components.

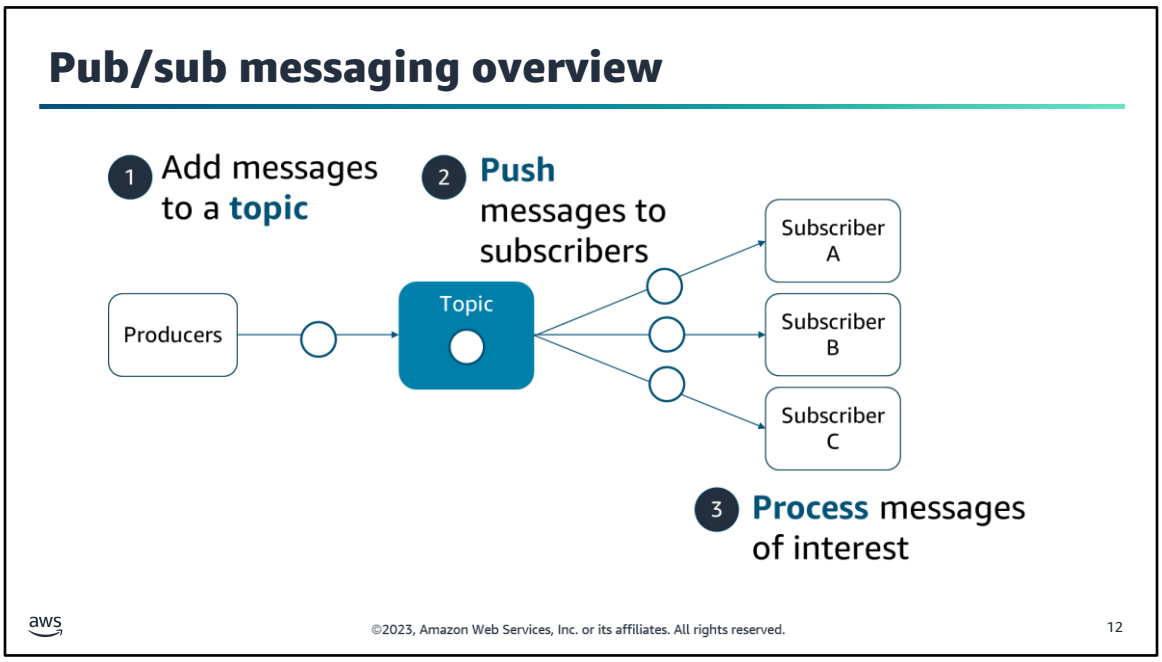
You'll learn about three fully managed services that you can use to implement each type of messaging service: Amazon Simple Queue Service (Amazon SQS) for message queues, Amazon Simple Notification Service (Amazon SNS) for pub/sub messaging, and Amazon Kinesis Data Streams for data streams.



A message queue is a temporary repository for messages that are waiting to be processed. Messages are usually small and can be requests, replies, error messages, or plain information. Examples include customer records, product orders, invoices, and patient records.

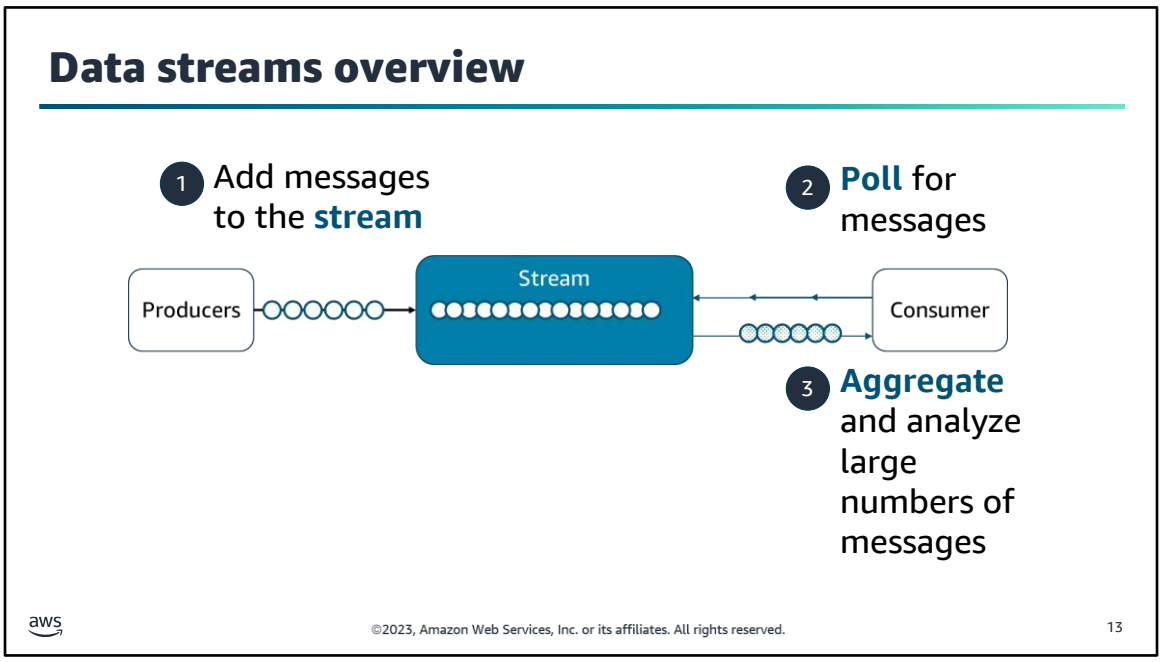
The following steps describe a message queue workflow:

1. A producer adds a message to the queue. The message is stored in the queue until another component (a consumer) retrieves and processes the message.
2. Consumers poll the queue to determine if new messages are available to process.
3. Each consumer processes each new message it retrieves.
4. If processing is successful, the consumer deletes the message from the queue so that no other consumer tries to process the same message.



With publish/subscribe (pub/sub) messaging, publishers (producers) send messages to topics. Message topics provide a lightweight mechanism to broadcast asynchronous event notifications. Topics also provide endpoints that software components can connect to.

Unlike queue consumers, subscribers do not need to check for messages in a pub/sub model. Messages that are published to a topic are pushed out to all topic subscribers. With pub/sub messaging, you can easily notify large numbers of subscribers about events. For example, you might publish a message to an "alerts" topic when a certain type of error occurs, and multiple consumers might take different actions on that message.

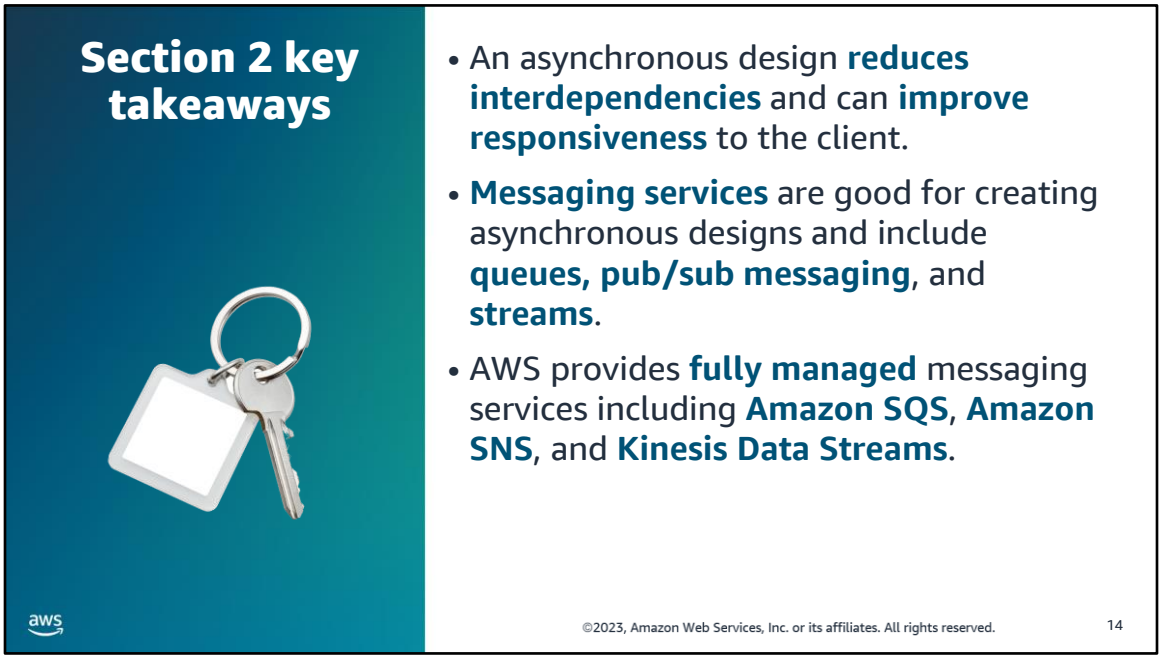


Data streams are similar to queues in that they provide a temporary repository for producer messages, and consumers poll the stream to check for new messages.

Streams are different from queues in that the rate of messages is typically much higher. Rather than processing each individual message on the queue, streams are typically used to analyze high volumes of data in near-real time.

Another distinction from queues is that multiple consumers might process the same messages and do entirely different actions with them. Consumers all view the same data (similar to looking through the same window) and act on the messages for their own purposes. Consumers do not delete messages from a stream.

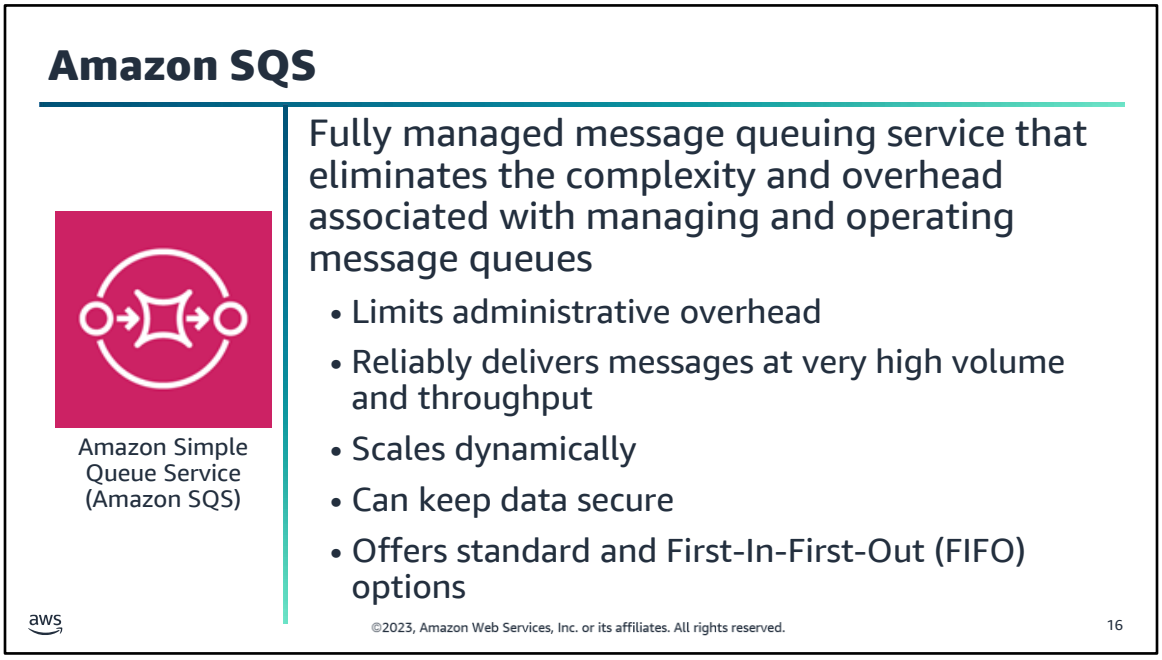
Streams are helpful for use cases where you need to handle rapid and continuous data intake and aggregation. For example, you might use a stream to process IT infrastructure log data, social media data, market data feeds, or web clickstream data.



The following are the key takeaways from this section of the module:

- An asynchronous design reduces interdependencies and can improve responsiveness to the client.
- Messaging services are good for creating asynchronous designs and include queues, pub/sub messaging, and streams.
- AWS provides fully managed messaging services including Amazon SQS, Amazon SNS, and Kinesis Data Streams.

Section 3: Introducing Amazon SQS.



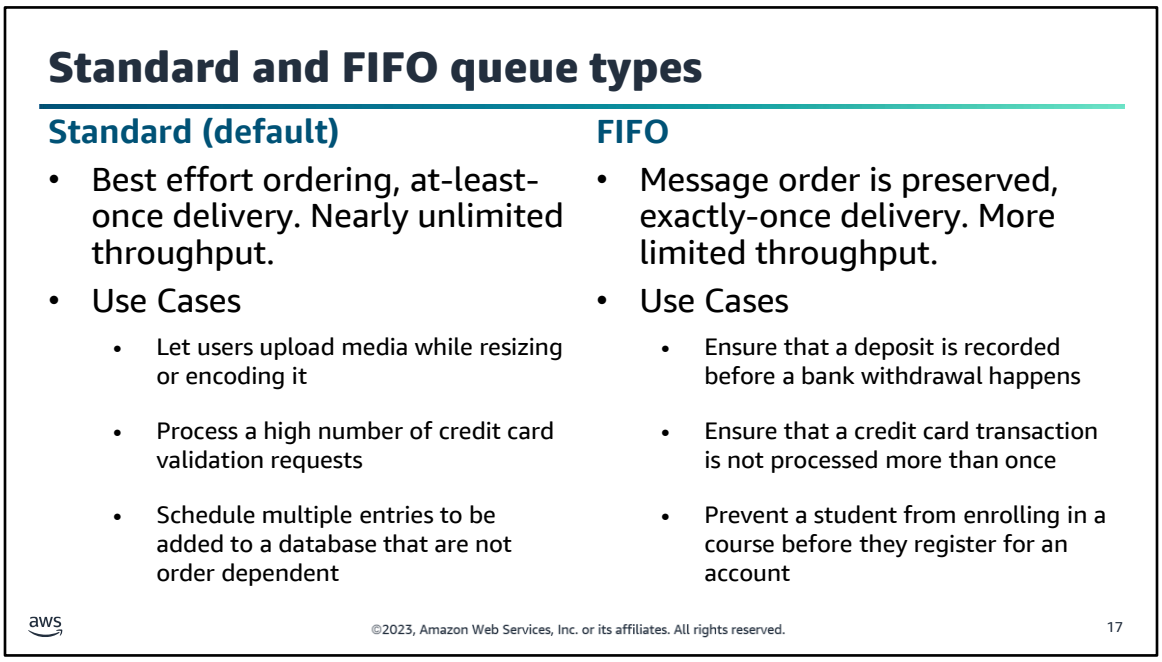
Amazon Simple Queue Service (Amazon SQS) is a fully managed message queuing service. With Amazon SQS, you can decouple and scale microservices, distributed systems, and serverless applications. The service eliminates the complexity and overhead associated with managing and operating message-oriented services, so that you can focus on the business logic of your application.

You can use Amazon SQS to transmit any volume of data at any level of throughput, without losing messages or requiring other services to be available. With Amazon SQS, you can decouple application components so that they run and fail independently, which increases the overall fault tolerance of the system. Multiple copies of every message are stored redundantly across multiple Availability Zones so that they are available when they are needed.

Amazon SQS uses the AWS Cloud to dynamically scale based on demand. Amazon SQS scales elastically with your application so that you don’t have to worry about capacity planning and pre-provisioning. The number of messages per queue is not limited.

You can use Amazon SQS to exchange sensitive data between applications using server-side encryption (SSE) to encrypt each message body. Amazon SQS SSE integration with AWS Key Management Service (AWS KMS) provides the ability to centrally manage the keys that protect Amazon SQS messages along with keys that protect your other AWS resources. AWS KMS logs every use of your encryption keys to AWS CloudTrail to help meet your regulatory and compliance needs.

Amazon SQS offers both standard and First-In-First-Out (FIFO) queues to support different processing priorities.



Amazon SQS supports two types of message queues:

- *Standard queues* are the default queue type. Standard queues provide best-effort ordering, which ensures that messages are generally delivered in the same order that they are sent. Standard queues support at-least-once message delivery. However, occasionally more than one copy of a message might be delivered out of order. Standard queues also support a nearly unlimited number of transactions per second (TPS) per action.
- *First-In-First-Out (FIFO) queues* are designed to enhance messaging between applications when the order of operations and events is critical, or where duplicates can't be tolerated. FIFO queues also provide exactly-once processing, but they have a limited number of TPS. The next section provides more information about queue type distinctions.

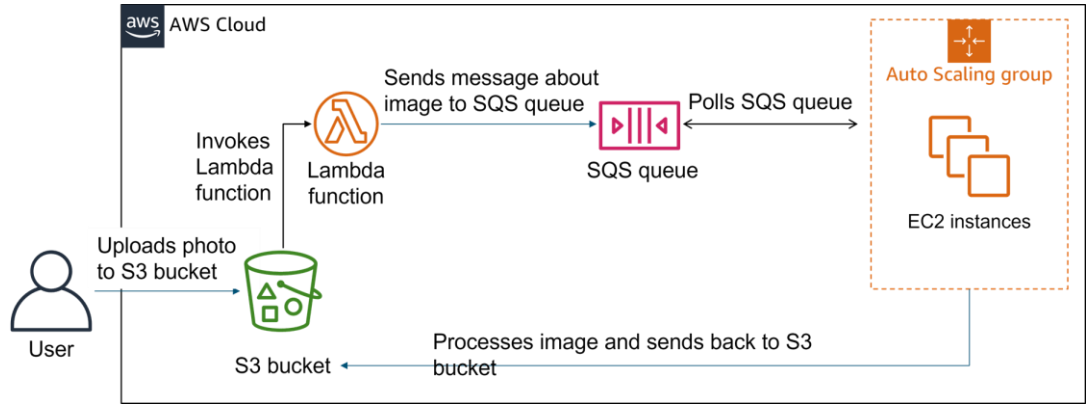
You can use standard message queues in many scenarios, as long as your application can process messages that arrive more than once and out of order. For example, you can use a standard queue for the following:

- Decouple live user requests from intensive background work: Enable users to upload media while resizing or encoding it.
- Allocate tasks to multiple worker nodes: Process a high number of credit card validation requests.
- Batch messages for future processing: Schedule multiple entries to be added to a database that are not order dependent and for which idempotency is not an issue.

You use FIFO queues for applications when the order of operations and events is critical, or in cases where duplicates can't be tolerated. For example, you can use a FIFO queue for the following:

- Bank transactions: Ensure that a deposit is recorded before a bank withdrawal happens.
- Credit card transactions: Ensure that a credit card transaction is not processed more than once.
- Course enrollment: Prevent a student from enrolling in a course before they register for an account.

Amazon SQS architecture example



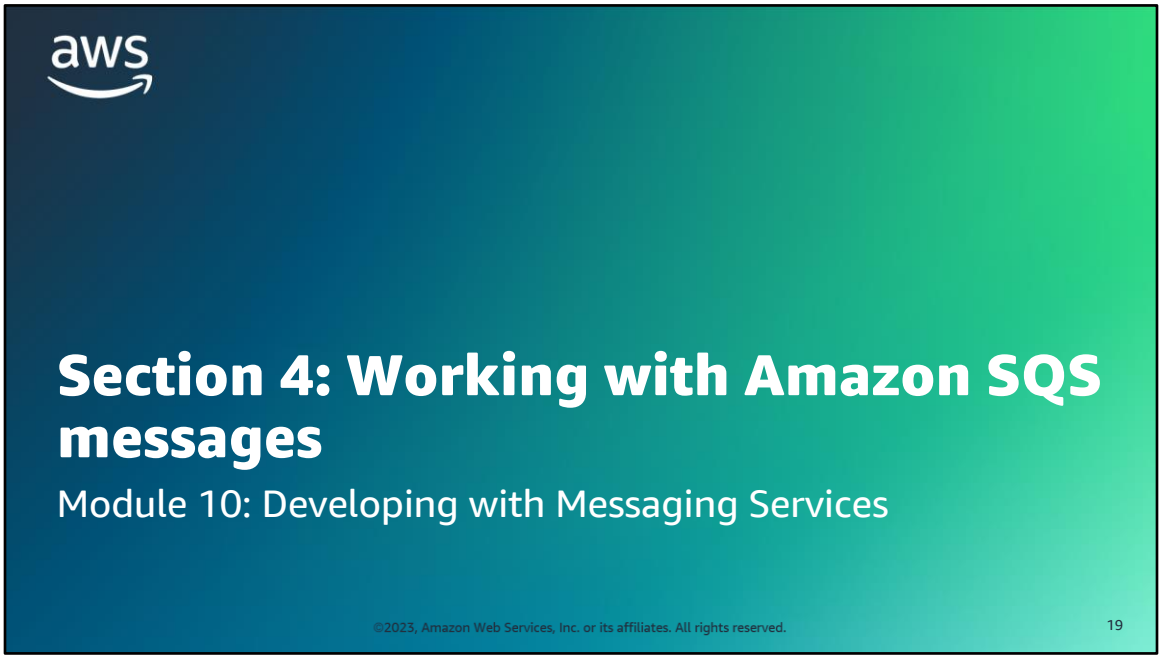
18

You can integrate Amazon SQS with other AWS services to make applications more reliable and scalable.

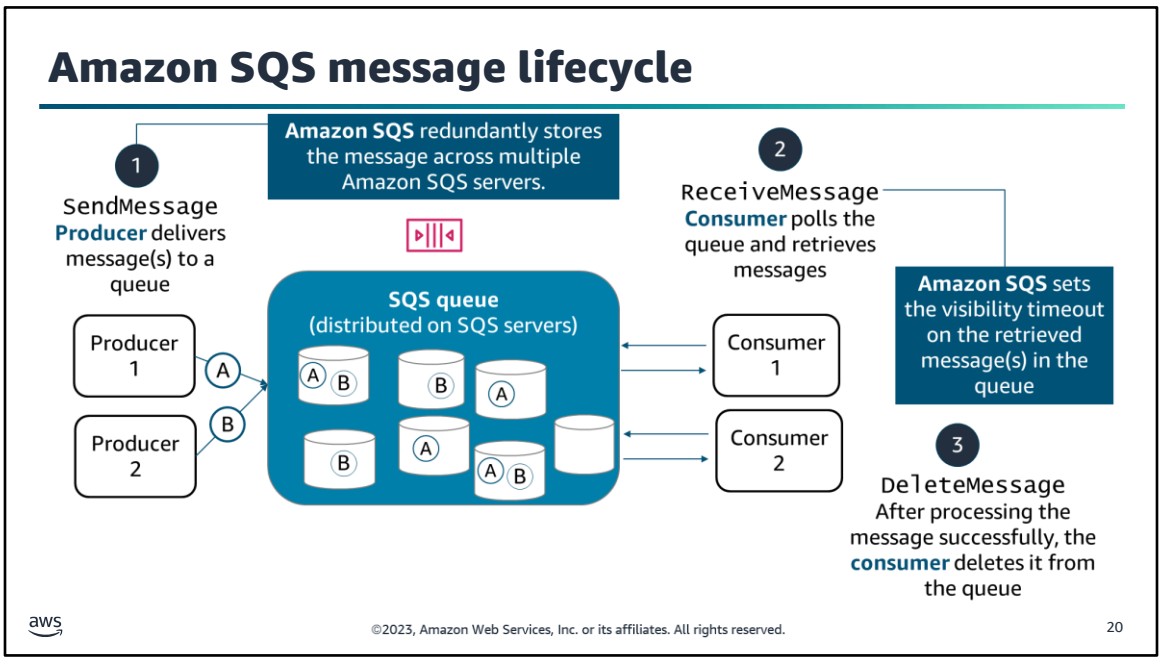
This slide illustrates an example of using an SQS queue to decouple components in an image processing application.

In this example, a user uploads a photo to an Amazon Simple Storage Service (Amazon S3) bucket, which invokes an AWS Lambda function. The Lambda function sends a message containing information about the image to an SQS queue. A processing server that is part of an Amazon EC2 Auto Scaling group polls the SQS queue for messages to process. After processing the photo, the processing server sends the processed photo back to the S3 bucket.

Note that the Auto Scaling group scales based on the size of the SQS queue. The processing servers keep working until the queue is empty and then keep holding until messages appear. If the EC2 instances become unavailable, the Lambda function can keep putting messages on the queue when an upload is made. When the Auto Scaling group is ready to start consuming messages again, it polls the queue for messages.



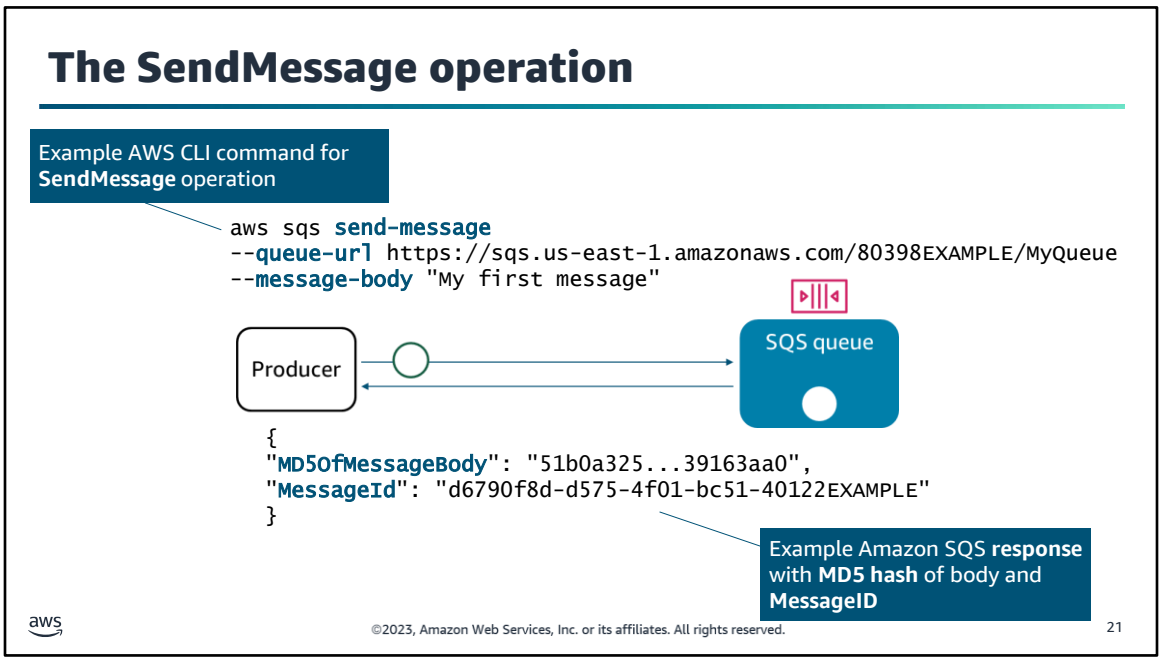
Section 4: Working with Amazon SQS messages.



The lifecycle of an Amazon SQS message is as follows:

1. A producer component sends a message to the SQS queue. Amazon SQS redundantly stores the message across multiple Amazon SQS servers.
2. A consumer component retrieves the message from the queue, and Amazon SQS starts the visibility timeout period. During the visibility timeout period, no other consumers can pick the message up from the queue.
3. The consumer component processes the message and then deletes it from the queue during the visibility timeout period. Note that if the message is not processed and deleted from the queue before the visibility timeout period expires, another consumer might pick up and process the same message.

You can perform these actions with the SendMessage, ReceiveMessage, and DeleteMessage API calls, respectively, either programmatically or from the Amazon SQS console. A variety of configuration options affect cost, message processing, and message retention.



The SendMessage operation delivers a message to a specific queue. This operation takes the following required request parameters:

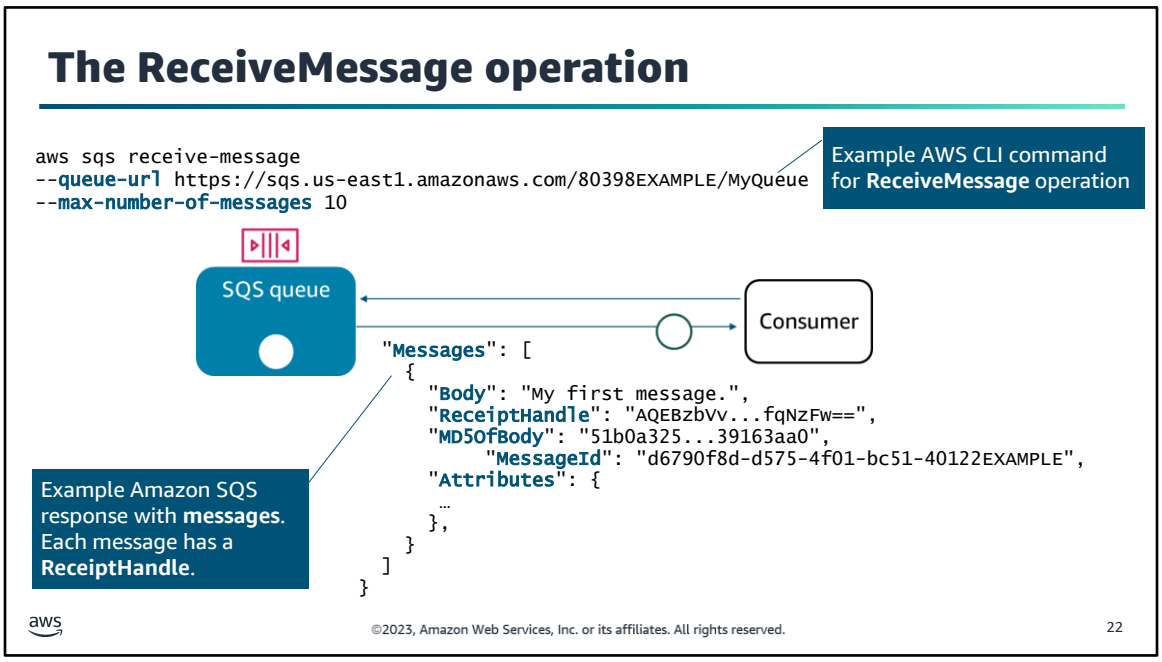
- **MessageBody (required):** The message to send. The maximum string size is 256 KB.
- **QueueUrl (required):** The URL of the Amazon SQS queue where a message is sent.

Optionally you might also include the following:

- **MessageAttribute (optional):** You can include structured metadata (such as timestamps, geospatial data, signatures, and identifiers) by using message attributes. Each message can have up to 10 attributes. Message attributes are separate from the message body; however, they are sent alongside it. Each message attribute consists of a name, type, and value.
- **DelaySeconds (optional):** The length of time, in seconds, to delay a specific message.

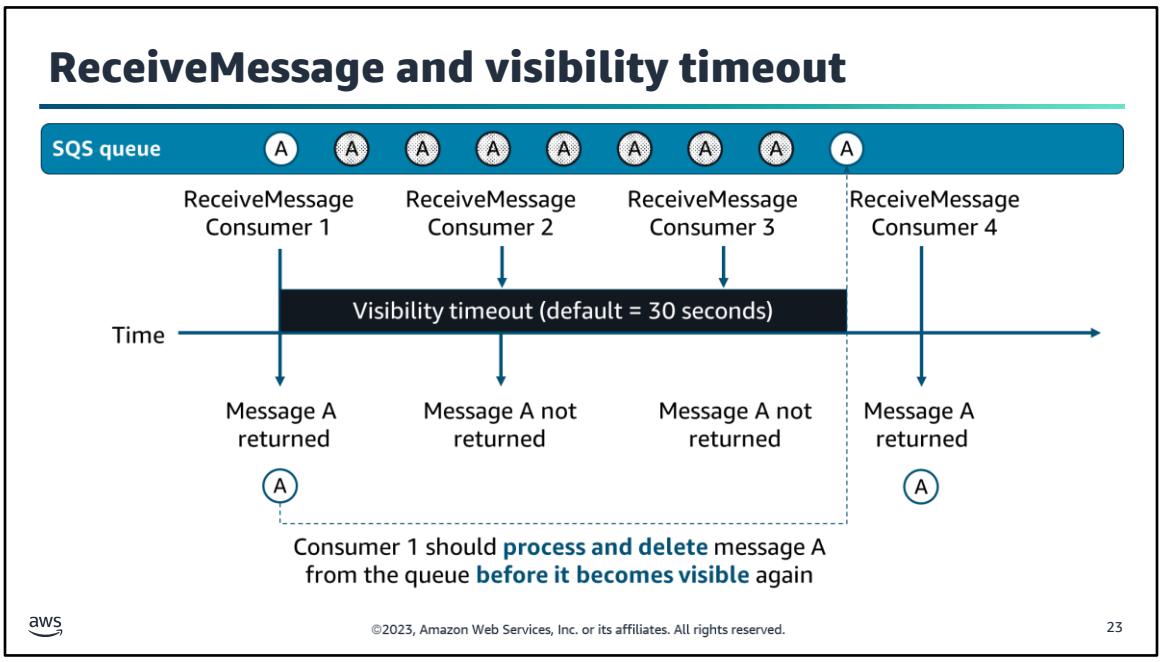
The slide illustrates how you might use the AWS Command Line Interface (AWS CLI) to call the SendMessage operation.

After you send a message, the Amazon SQS response includes a Message-Digest algorithm 5 (MD5) digest of the message body string, which you can use to verify that Amazon SQS received the message correctly. The response also includes a system-assigned MessageId, which is useful for identifying messages. Additional elements might be included in the response depending on the message parameters and the type of queue.



You can retrieve one or more messages (up to 10) from the queue by using the ReceiveMessage operation. You set the number of messages that you want to retrieve by using the MaxNumberOfMessages parameter.

Amazon SQS responds with a message or messages depending on the MaxNumberOfMessages that you set in the ReceiveMessage command as well as the number of records currently in queue. For each record in the "Messages" response, Amazon SQS returns a *receipt handle* for that message. This handle is associated with the action of receiving the message, not with the message itself. If you receive a message more than once, you get a different receipt handle each time that you receive it. You need the receipt handle to delete the message or change its visibility on the queue.



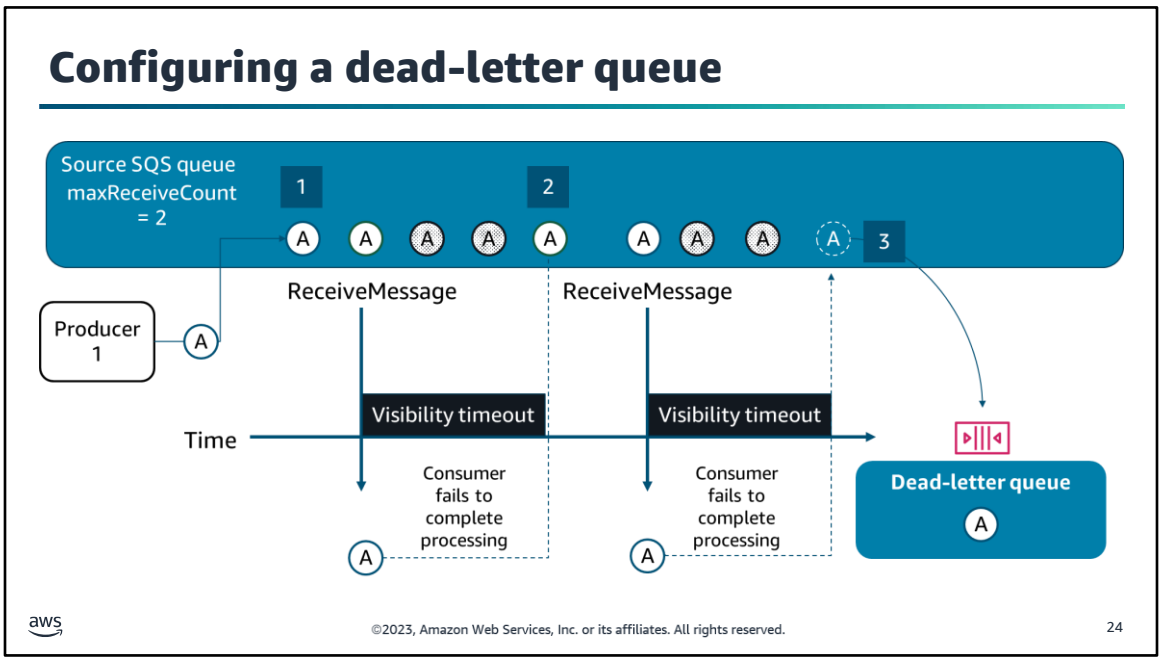
Immediately after a message is received using the ReceiveMessage operation, Amazon SQS sets a visibility timeout. The visibility timeout is a period during which other consumers cannot receive and process the message.

Before the visibility timeout expires, the consumer is expected to process AND then delete the message from the queue. Amazon SQS doesn't automatically delete the message because, as a distributed system, there's no guarantee that the consumer actually received and handled the message (for example, because of a connectivity issue or because of an issue in the consumer application).

For the duration of the visibility timeout, Amazon SQS temporarily stops returning the message as part of any ReceiveMessage requests.

The visibility timeout defaults to 30 seconds. The minimum is 0 seconds, and the maximum is 12 hours. You can use the VisibilityTimeout parameter on the ReceiveMessage operation to modify the visibility timeout for a ReceiveMessage request.

If your consumer doesn't delete the message before the visibility timeout expires, the message becomes visible to other consumers, and Amazon SQS flags the message as being returned to the queue. Amazon SQS increments its counter each time the message becomes visible on the queue. By default, Amazon SQS will increment the counter and make the message available to consumers until the retention period for that message expires. You can modify this behavior using a dead-letter queue with a redrive policy.



For messages that cannot be processed (and deleted by the consumer), configure a dead-letter queue in Amazon SQS. Dead-letter queues can help you troubleshoot incorrect transmission operations and stop your queue from continually trying to process a corrupted message.

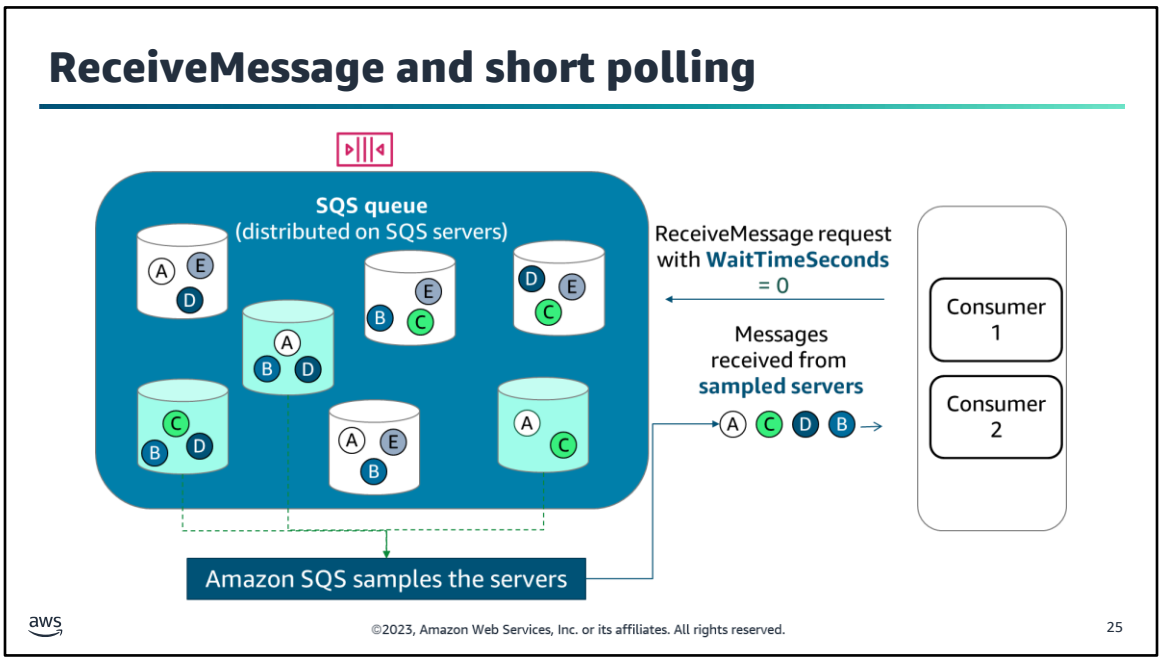
The dead-letter queue can receive messages from the *source queue* after the maximum number of processing attempts (maxReceiveCount) has been reached. The maxReceiveCount is set as part of the source queue's redrive policy. Messages can be sent to and received from a dead-letter queue just like any other SQS queue. You can create a dead-letter queue from the Amazon SQS API or the Amazon SQS console.

To configure the dead-letter queue from the console, first create a queue to be used as your dead-letter queue. Then, on the source queue, choose to configure a redrive policy. As part of the redrive policy, select the queue to use as the dead-letter queue, and set the maximum receives option to tell Amazon SQS how many processing attempts to make before sending the message to the dead-letter queue.

In the example, on the slide the source queue is set with a redrive policy with a maxReceiveCount equal to two. Because of the policy, message A is moved to the dead-letter queue after the second consumer fails to process the message. Rather than making the message visible to consumers for a third time, Amazon SQS moves message A to the dead-letter queue, and the message is no longer available in the source queue.

For more information, see the following resources in the *Amazon SQS Developer Guide*:

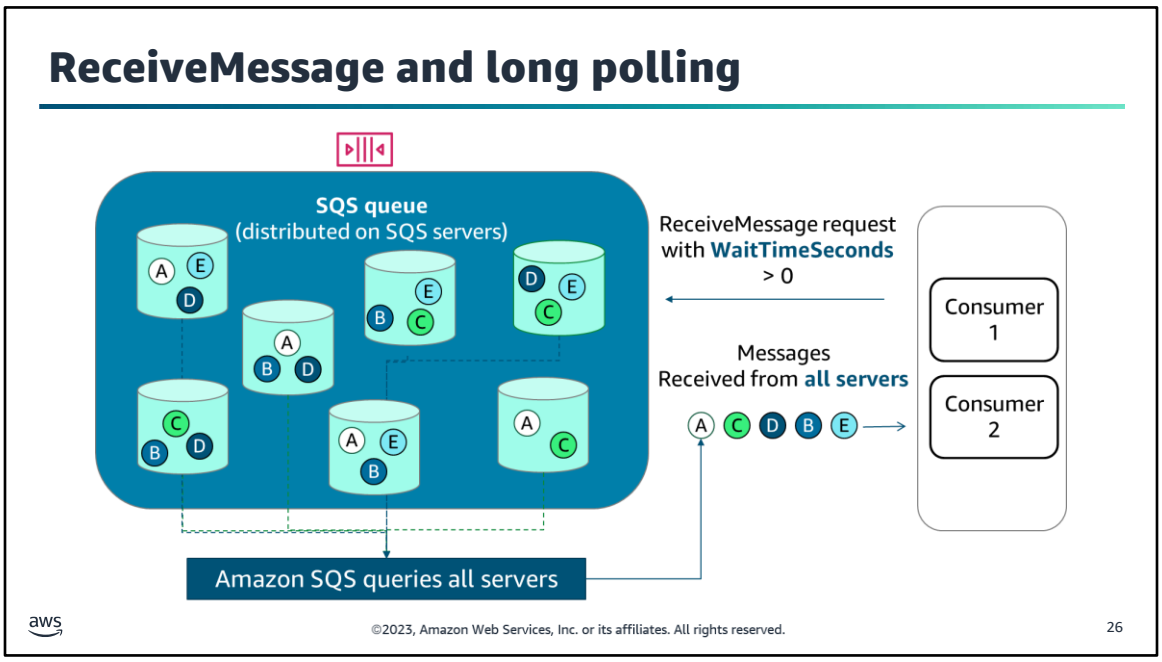
- Amazon SQS Dead-Letter Queues: https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html.
- Configuring a Dead-Letter Queue (Console): https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-configure-dead-letter-queue.html.



As noted earlier, consumers use a polling model to retrieve messages from an SQS queue. By default, when you make a ReceiveMessage API call, Amazon SQS performs *short polling*, in which it samples a *subset* of SQS servers (based on a weighted random distribution).

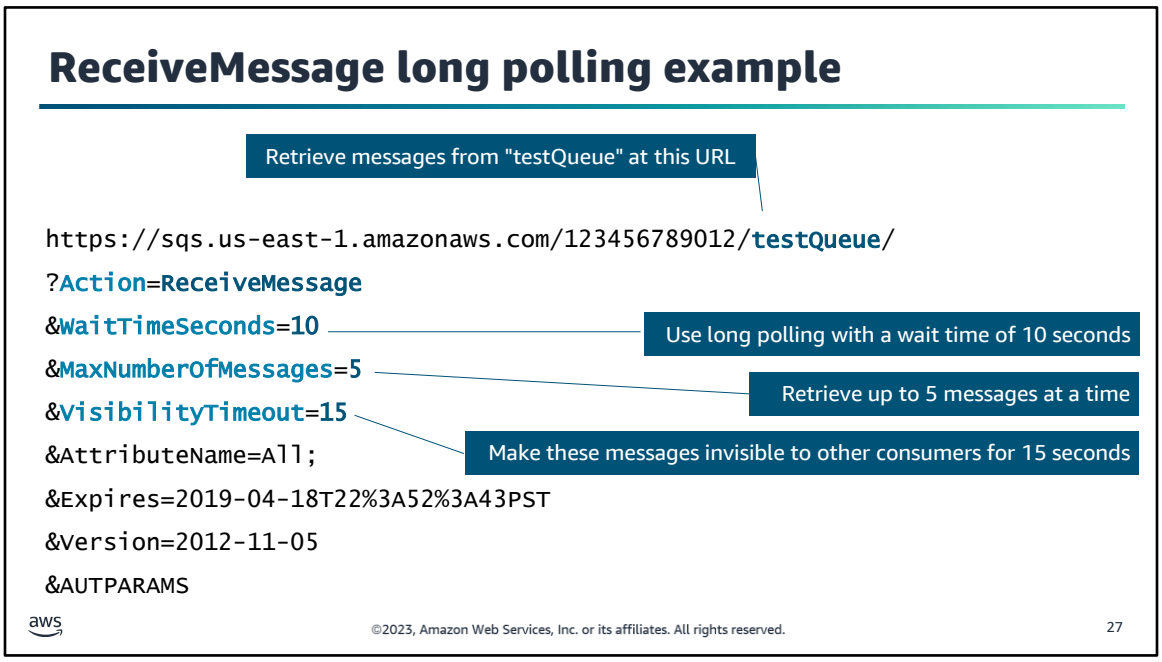
Amazon SQS immediately returns only the messages that are on the sampled servers. In the example, short polling returns messages A, B, C, and D, but it does not return message E. Short polling occurs when the WaitTimeSeconds parameter of a ReceiveMessage call is set to 0.

If the number of messages in the queue is small (fewer than 1,000), you will most likely get fewer messages than you requested per ReceiveMessage call. If the number of messages in the queue is extremely small, you might not receive any messages from a particular ReceiveMessage call. If you keep requesting ReceiveMessage, Amazon SQS will sample all of the servers and you will eventually receive all of your messages.



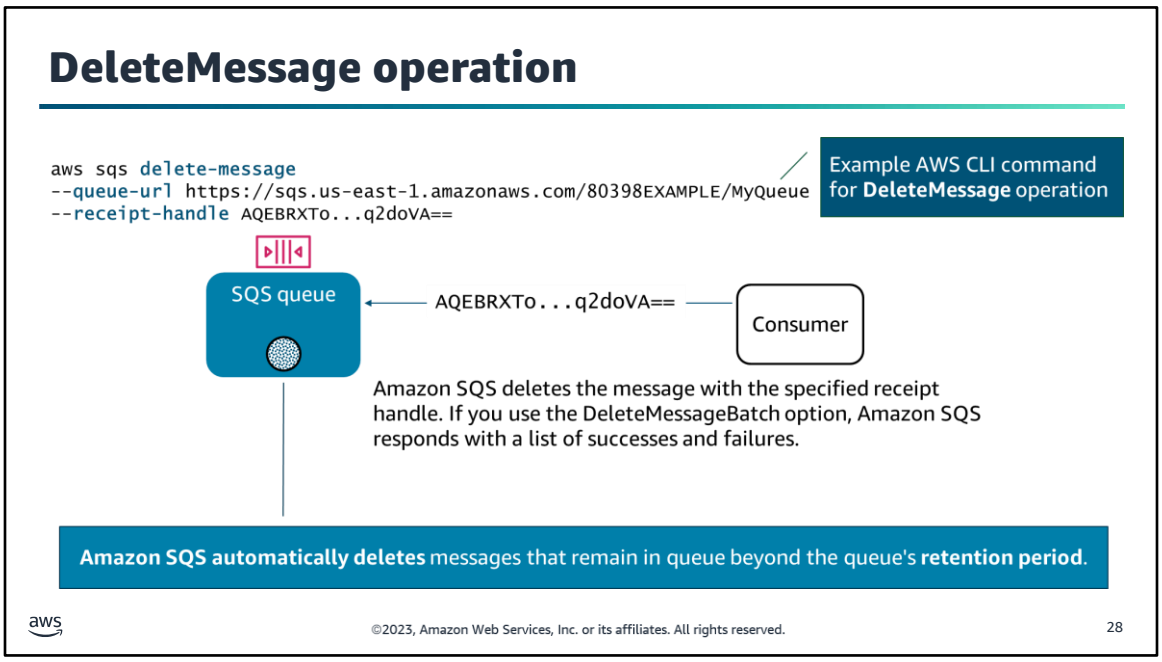
By contrast, with *long polling*, Amazon SQS queries *all* of the servers and waits until a message is available in the queue before it sends a response. Long polling helps reduce the cost of using Amazon SQS by eliminating the number of empty responses (when no messages are available for a ReceiveMessage request) and false empty responses (when messages are available but aren't included in a response).

To enable long polling, set the WaitTimeSeconds parameter of the ReceiveMessage request to a non-zero value between 1 and 20 seconds.



The slides provides an example of a query to retrieve messages from a queue named testQueue by using the ReceiveMessage API. The query includes the following parameters to modify the default behaviors:

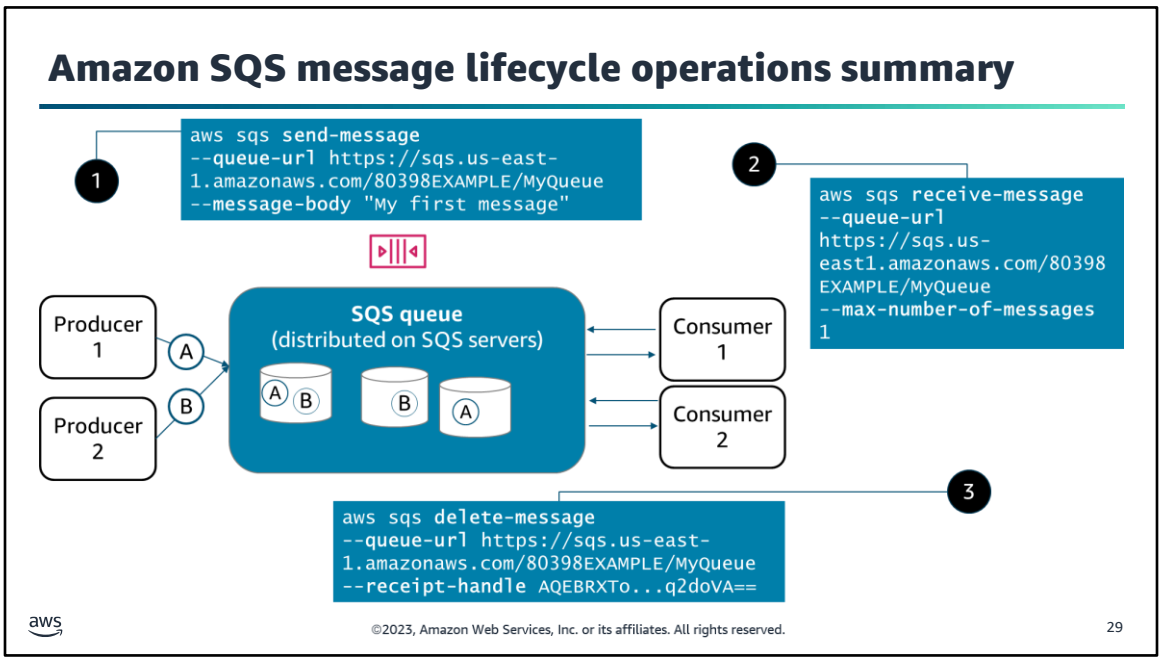
- **WaitTimeSeconds** is set to 10, which indicates that long polling is enabled.
- **MaxNumberOfMessages** is set to 5, which indicates the maximum number of messages to return in a single call.
- **VisibilityTimeout** is set to 15, which indicates that the message will be invisible to other consumers to process for 15 seconds.



To prevent a message from being received and processed again when the visibility timeout expires, the consumer must delete the message. You can use the DeleteMessage operation to delete a specific message from a specific queue, or you can use DeleteMessageBatch to delete up to 10 messages. To select the message to delete, use the receipt handle of the message. To use the DeleteMessageBatch operation, provide the list of receipt handles to be deleted.

The ReceiptHandle is associated with a specific instance of receiving a message. If you receive a message more than once, the ReceiptHandle is different each time that you receive a message. When you use the DeleteMessage action, you must provide the most recently received ReceiptHandle for the message.

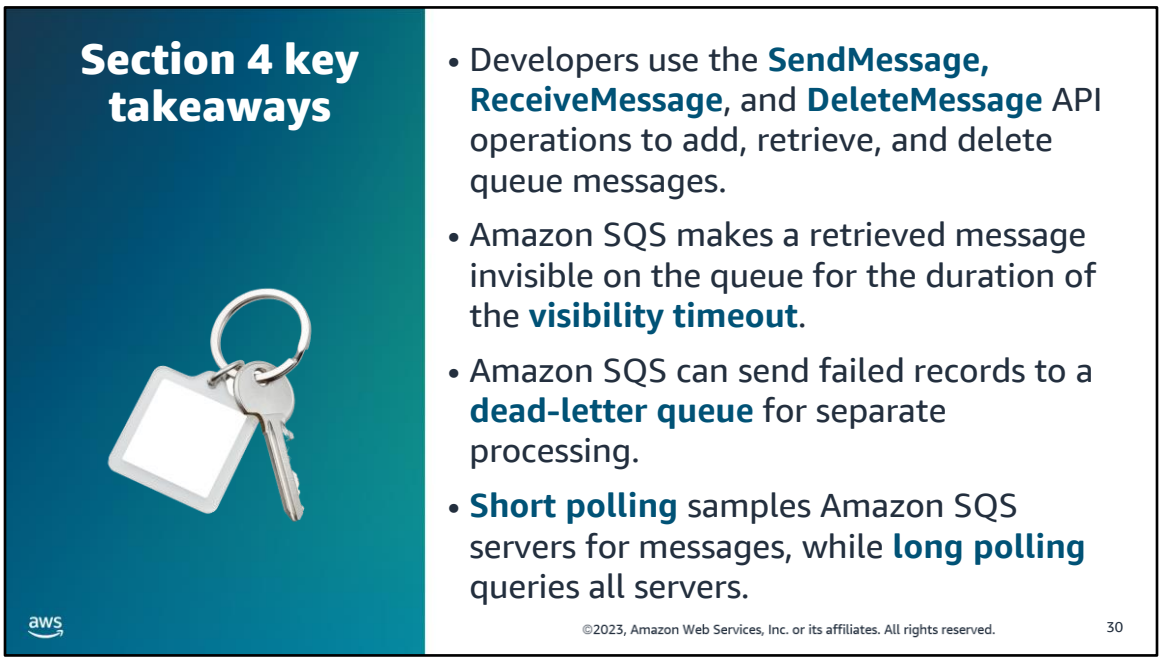
Amazon SQS automatically deletes messages that have been in a queue longer than the queue’s configured message retention period. The default message retention period is 4 days. However, you can set the message retention period to a value from 60 seconds to 1,209,600 seconds (14 days) by using the SetQueueAttributes action.



The lifecycle of an Amazon SQS message is as follows:

1. A producer component sends a message to the SQS queue. Amazon SQS redundantly stores the message across multiple Amazon SQS servers.
2. A consumer component retrieves the message from the queue, and Amazon SQS starts the visibility timeout period. During the visibility timeout period, no other consumers can pick the message up from the queue.
3. The consumer component processes the message and then deletes it from the queue during the visibility timeout period. Note that if the message is not processed and deleted from the queue before the visibility timeout period expires, another consumer might pick up and process the same message.

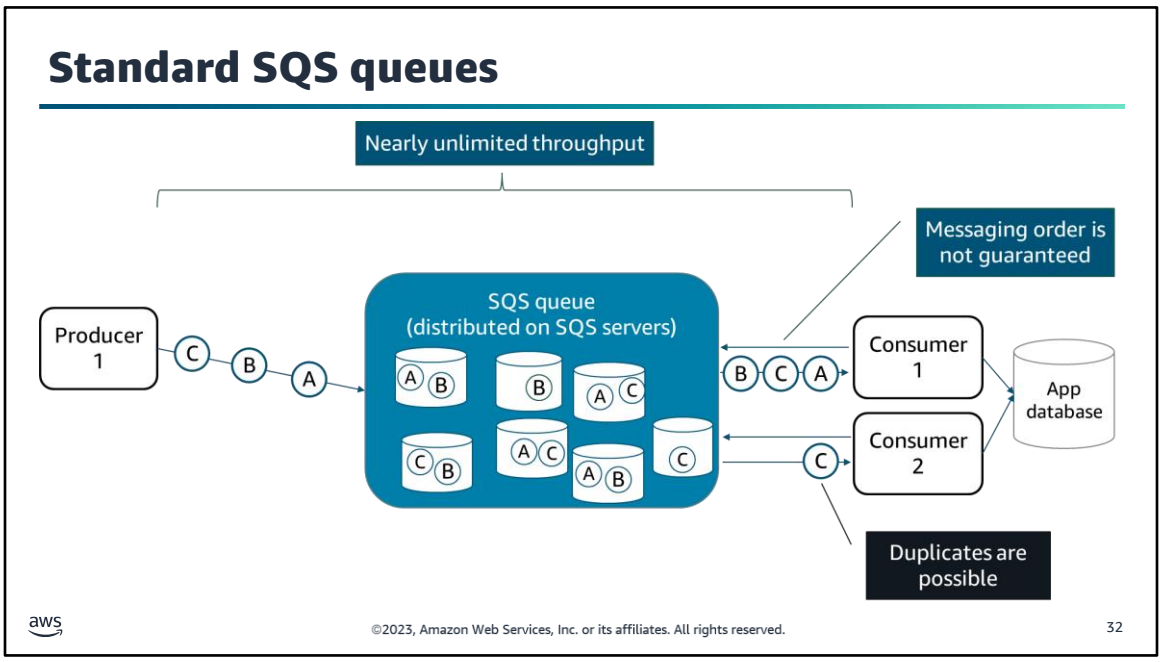
You can perform these actions with the SendMessage, ReceiveMessage, and DeleteMessage API calls, respectively, either programmatically or from the Amazon SQS console. A variety of configuration options affect cost, message processing, and message retention.



The following are the key takeaways from this section of the module:

- Developers use the SendMessage, ReceiveMessage, and DeleteMessage API operations to add, retrieve, and delete queue messages.
- Amazon SQS makes a retrieved message invisible on the queue for the duration of the visibility timeout.
- Amazon SQS can send failed records to a dead-letter queue for separate processing.
- Short polling samples Amazon SQS servers for messages, while long polling queries all servers.

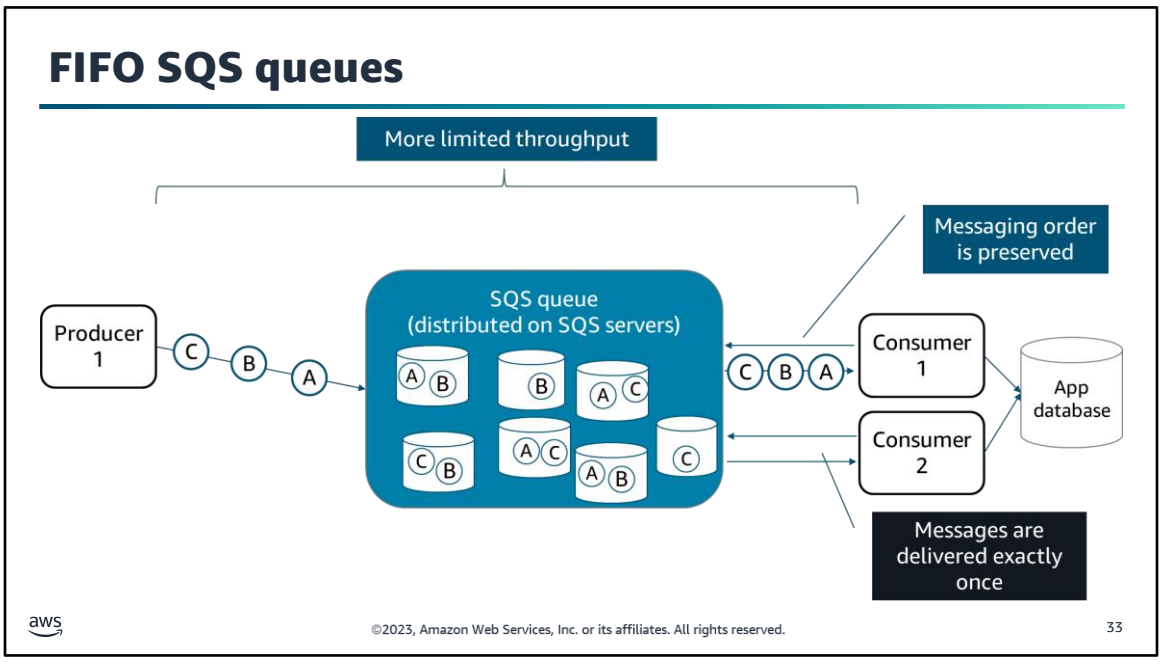
Section 5: Configuring Amazon SQS queues.



Standard queues are the default queue type. Standard queues support a nearly unlimited number of transactions per second (TPS) per action.

Because of the queue's highly distributed architecture (which allows this high throughput), you need to account for the following two potential conditions related to message delivery to your consumer(s).

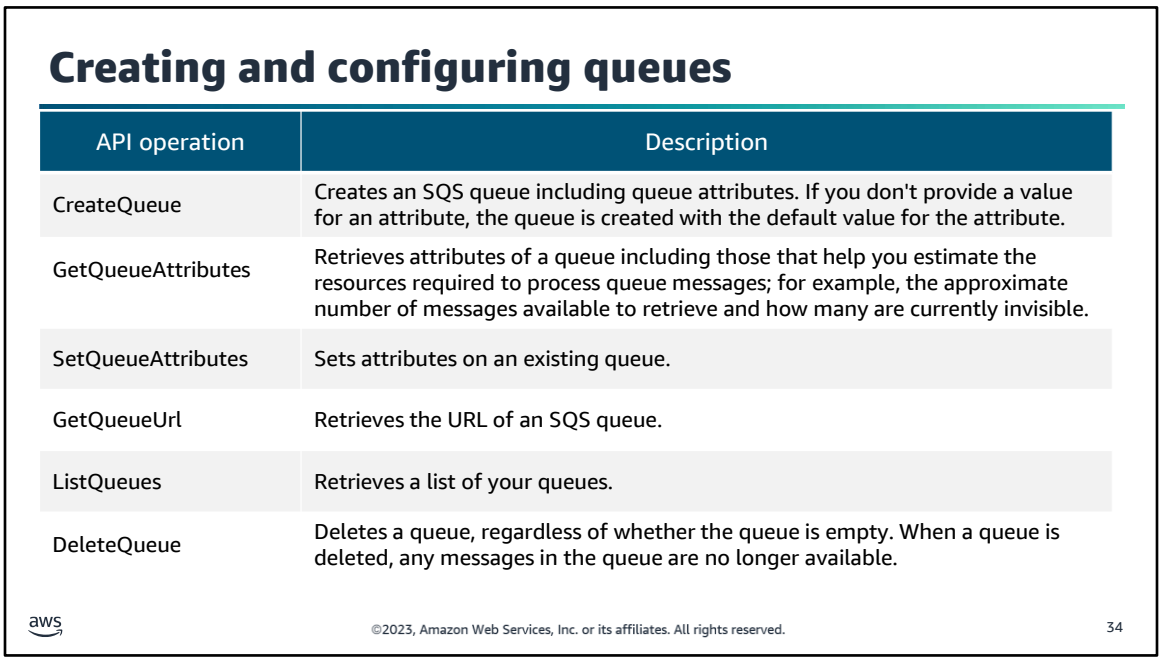
1. **Messaging order is not guaranteed.** Standard queues provide best-effort ordering. This ensures that messages are generally delivered in the same order as they are sent, but some might be delivered out of order. If you use a standard queue and have a target application that needs to process messages in the order received, you need to include logic in your consumer application to reorder messages before further processing; for example, using the timestamp of each message to compare and order them.
2. **More than one copy of a message might be delivered**. Standard queues support at-least-once delivery, not exactly-once delivery. To avoid processing the same message more than once, your application code needs to check for duplicates and only process a message the first time it is received. For example, you might add code that checks your database for a unique value in the incoming event and ignores it if it already exists in the database. MessageID is a unique identifier that Amazon SQS generates. If you want to verify the uniqueness of the record on the SQS queue, you could use the messageID attribute. Another scenario might be that you actually need to verify the uniqueness of the payload. For example, if an upstream producer sent the same record to Amazon SQS twice, each event would have a unique messageID in Amazon SQS but would be the same payload from your application’s perspective. You can avoid processing the same payload twice using the md5OfBody attribute in the Amazon SQS event. This field represents a unique hash of the payload.



To avoid having to write code to handle message ordering and deduplication where order is critical and duplicates can't be tolerated, use a First-In-First-Out (FIFO) queue.

FIFO queues are designed to provide exactly-once processing, which helps you to avoid sending duplicates to a queue. FIFO queues also guarantee message ordering. FIFO queues support *message groups*, which allow multiple ordered message groups within a single queue.

FIFO queues do have a more limited transaction per second (TPS) throughput than standard queues, so only choose a FIFO queue if you need the features of a FIFO queue.



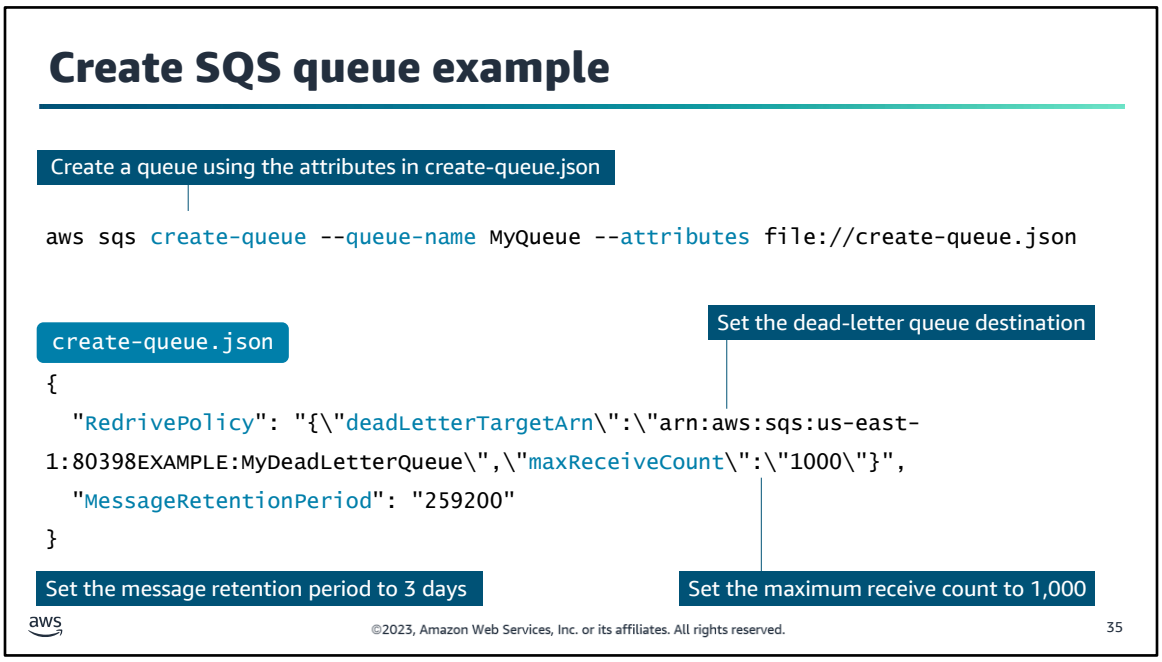
You can use the AWS Management Console to create and configure your SQS queues, and you might find it helpful to use the console to send test messages to your queue.

You can also create and configure queues using API operations, and set queue parameters similar to some of the message parameters that you just learned about. For example, you can set the WaitTimeSeconds on the queue itself rather than an individual ReceiveMessage request.

The following are the basic queue options:

- **CreateQueue:** Create an SQS queue including queue attributes.
- **SetQueueAttributes** and **GetQueueAttributes:** Set and retrieve the attributes of a queue, respectively. These options help you estimate the resources required to process Amazon SQS messages.
- **GetQueueUrl:** Retrieve the URL of an SQS queue.
- **ListQueues:** Retrieve a list of your queues.
- **DeleteQueue:** Deletes a queue, regardless of whether the queue is empty. When a queue is deleted, any messages in the queue are no longer available.

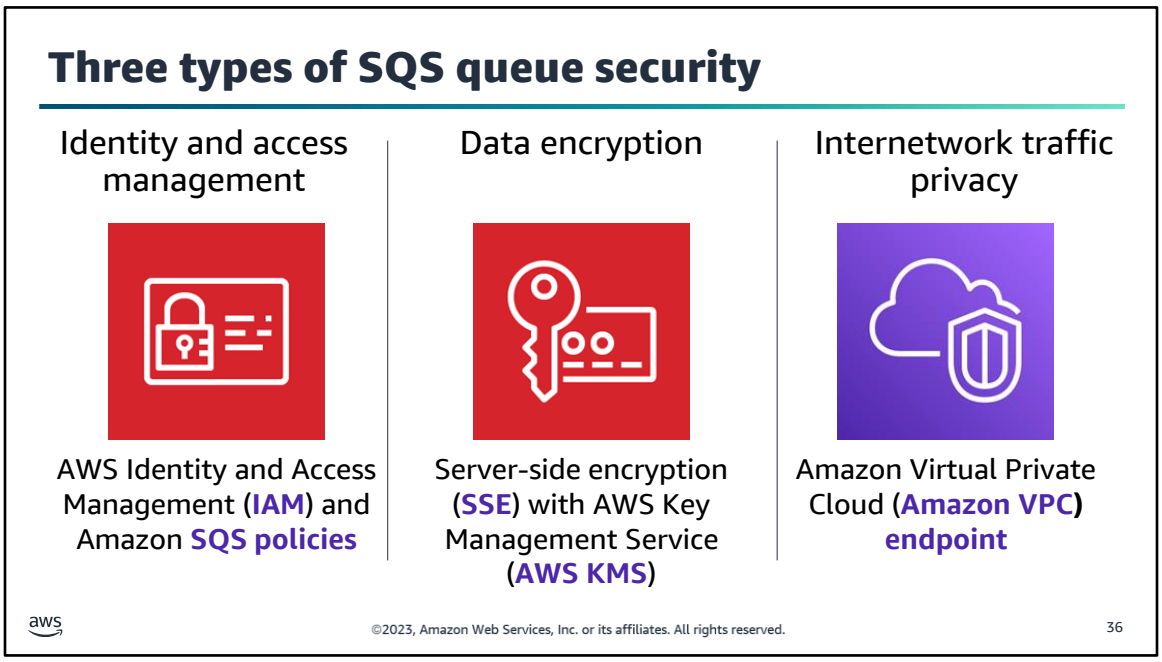
For more information, see the Amazon SQS API Reference at https://docs.aws.amazon.com/AWSSimpleQueueService/latest/APIReference/Welcome.html.



This AWS CLI example creates a queue named MyQueue with the parameters in the create-queue.json file.

The create-queue.json file specifies a redrive policy for failed records. The policy sends failures to a dead-letter queue called MyDeadLetterQueue after 1,000 attempts (maxReceiveCount).

The parameters file also sets the message retention period for the source queue to 259,200 seconds or 3 days. If a message remains on the source queue for more than 3 days, it will be discarded regardless of the maxReceiveCount.



There are three ways to secure your Amazon SQS resources:

- **For identity and access management:** Access to Amazon SQS requires credentials that AWS can use to authenticate your requests. These credentials must have permissions to access AWS resources, such as SQS queues and messages. Use AWS Identity and Access Management (IAM) policies and Amazon SQS policies. Amazon SQS has its own resource-based permissions system that uses policies that are written in the same language that is used for IAM policies. Thus, you can achieve similar results with Amazon SQS policies and IAM policies, and you might use them in combination. An example is presented on the next slide. The main difference is that you MUST use SQS policies to grant permissions to other AWS accounts.

  For more information about using IAM and Amazon SQS policies, see the following sections of the *Amazon SQS Developer Guide*:
  - Overview of Managing Access in Amazon SQS: https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-overview-of-managing-access.html
  - Using Custom Policies with the Amazon SQS Access Policy Language: https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-creating-custom-policies.html

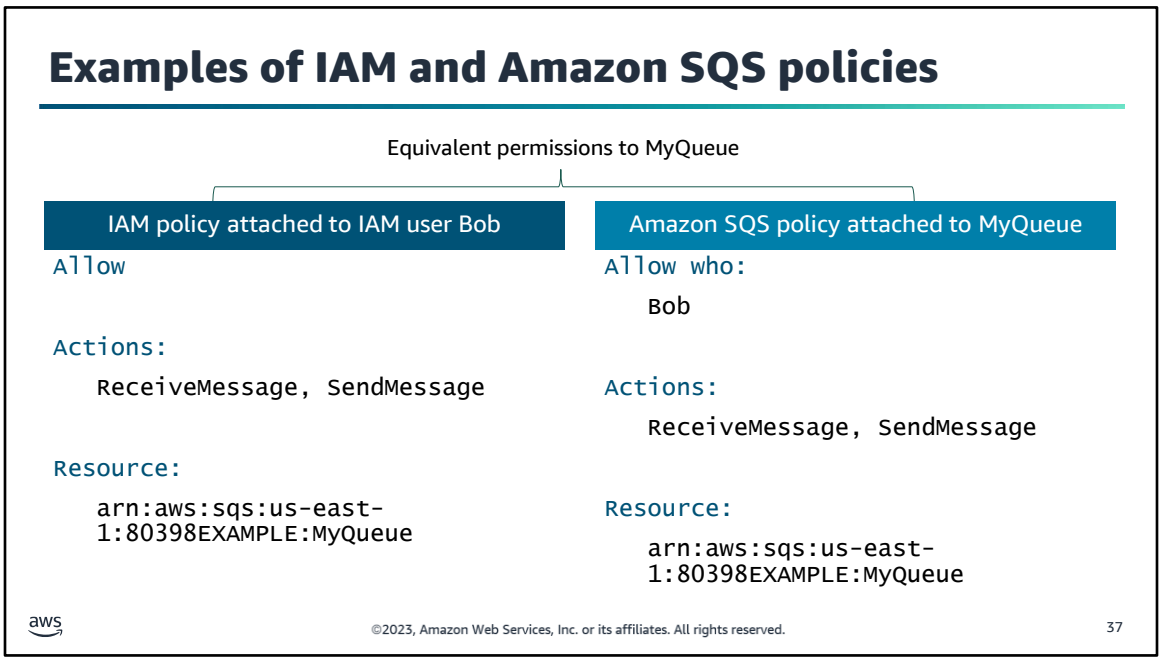
- **For data encryption:** Encrypt your data using server-side encryption (SSE). With SSE, you can transmit sensitive data in encrypted queues. SSE protects the contents of messages in SQS queues by using keys that are managed by AWS KMS. When you enable SSE on an SQS queue, Amazon SQS will encrypt an incoming message before storing it on the queue and unencrypt it upon delivery to the consumer. Note that the encryption occurs at a point in time and applies to messages arriving *after* that point. For example, if messages 1–10 are on the queue when you enable SSE, message 11 will be encrypted, but messages 1–10 will not be.

For more information, see the Encryption at Rest section of the *Amazon SQS Developer Guide* at https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-server-side-encryption.html.

- **For internetwork traffic privacy:** If you use Amazon Virtual Private Cloud (Amazon VPC) to host your AWS resources, you can establish a connection between your VPC and Amazon SQS. You can use this connection to send messages to your SQS queues without crossing the public internet.

  For more information, see the Internetwork Traffic Privacy section of the *Amazon SQS Developer Guide* at https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-internetwork-traffic-privacy.html.

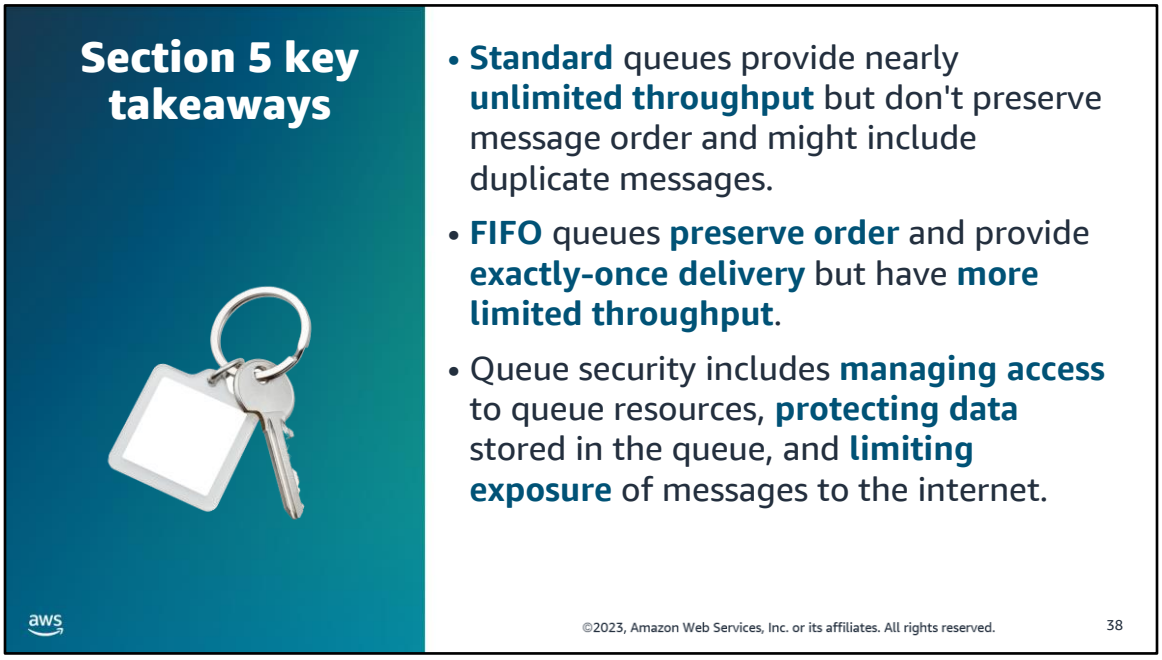
For more information about Amazon SQS security, see the *Amazon SQS Developer Guide* at https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-security.html.



In this example, the IAM policy and Amazon SQS policy grant equivalent access to a user named Bob who is in the account where the queue exists.

The IAM policy grants the permissions to use the Amazon SQS ReceiveMessage and SendMessage actions for the queue called MyQueue in the AWS account. The policy is attached to Bob.

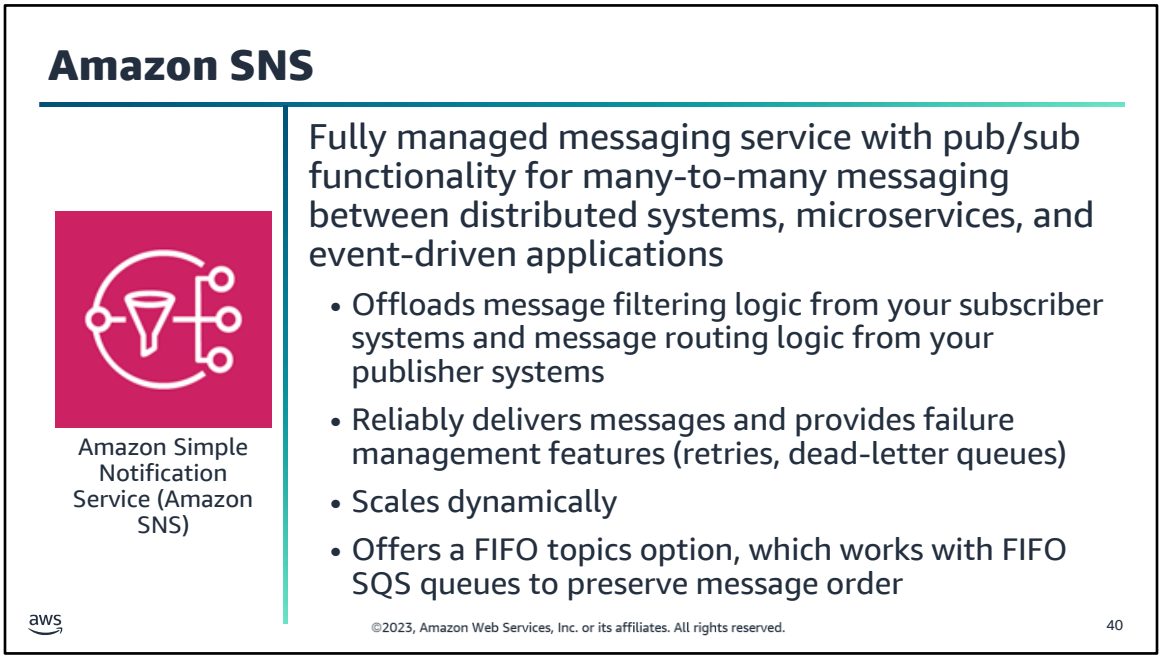
The Amazon SQS policy also gives permissions to Bob to use the ReceiveMessage and SendMessage actions for the same queue.



The following are the key takeaways from this section of the module:

- Standard queues provide nearly unlimited throughput but don't preserve message order and might include duplicate messages.
- FIFO queues preserve order and provide exactly-once delivery but have more limited throughput.
- Queue security includes managing access to queue resources, protecting data stored in the queue, and limiting exposure of messages to the internet.

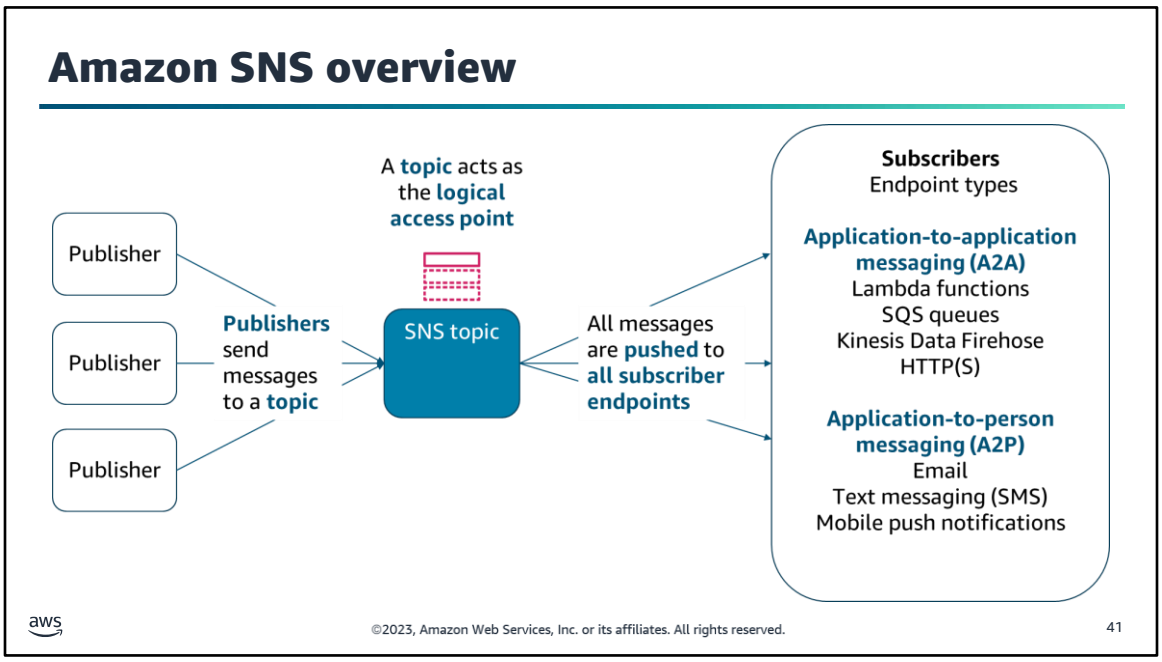
Section 6: Introducing Amazon SNS.



Amazon Simple Notification Service (Amazon SNS) is a highly available, durable, secure, fully managed pub/sub messaging service. With Amazon SNS, you can decouple microservices, distributed systems, and serverless applications. The service provides topics for high-throughput, push-based, many-to-many messaging.

Amazon SNS includes substantial retry policies to limit the potential for delivery failures and provides a dead-letter queue option for messages that continue to fail.

Amazon SNS also offers a First-In-First-Out (FIFO) topics option. You can use this option with FIFO SQS queues to ensure message ordering without writing a lot of custom code to manage message ordering and deduplication.



An SNS topic is a logical access point, which acts as a communication channel. Instead of including a specific destination address in each message, a publisher sends a message to the topic. Amazon SNS matches the topic to a list of subscribers for that topic. The service then delivers the message to each subscriber.

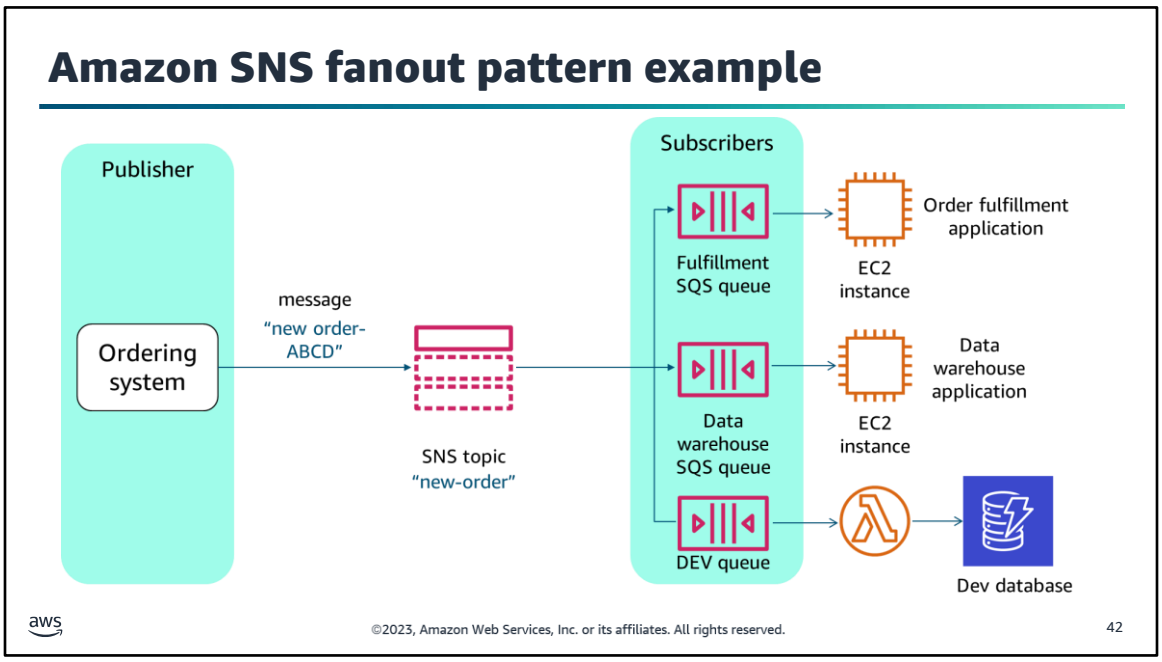
Each topic has a unique name, which identifies the Amazon SNS endpoint for publishers to post messages and subscribers to register for notifications.

A publisher can send messages to topics that they have created or to topics they have permissions to publish to. When you create an SNS topic, you can control access to it by defining policies that determine which publishers and subscribers can communicate with the topic.

To receive messages that are published to a topic, you must subscribe an endpoint to the topic. When you subscribe an endpoint to a topic, the endpoint begins to receive messages published to the associated topic. Subscribers receive all messages that are published to the topics that they subscribe to, and all subscribers to a topic receive the same messages. Amazon SNS formats the message based on each endpoint type.

Amazon SNS supports application-to-application messaging (A2A) with Lambda functions, SQS queues, Amazon Kinesis Data Firehose delivery streams, and HTTP(S). Amazon SNS supports application-to-person messaging (A2P) using email, text messaging (also known as short message service or SMS), and mobile push notifications.

For more information about using Amazon SNS for application-to-application (A2A) messaging, see the *Amazon SNS Developer Guide* at https://docs.aws.amazon.com/sns/latest/dg/sns-system-to-system-messaging.html.

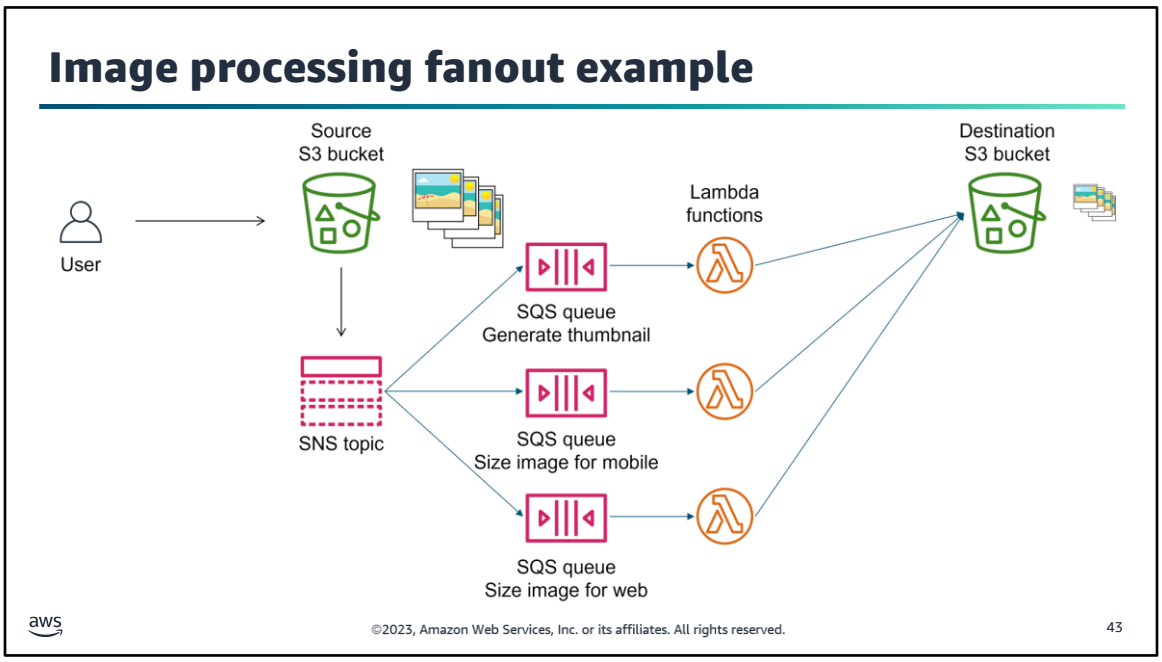


When a message must be processed by more than one consumer, you can combine pub/sub messaging with a message queue in a *fanout design pattern*. In a fanout pattern, you use an SNS topic to receive a message that is then pushed out to multiple subscribers for parallel processing.

In the example on the slide, SQS queues subscribe to the "new-order" topic. When a message is pushed from that topic, each queue gets an identical message but performs its own asynchronous processing. In this example, one queue hands off messages to an order fulfillment application, and the other queue interacts with a data warehousing application.

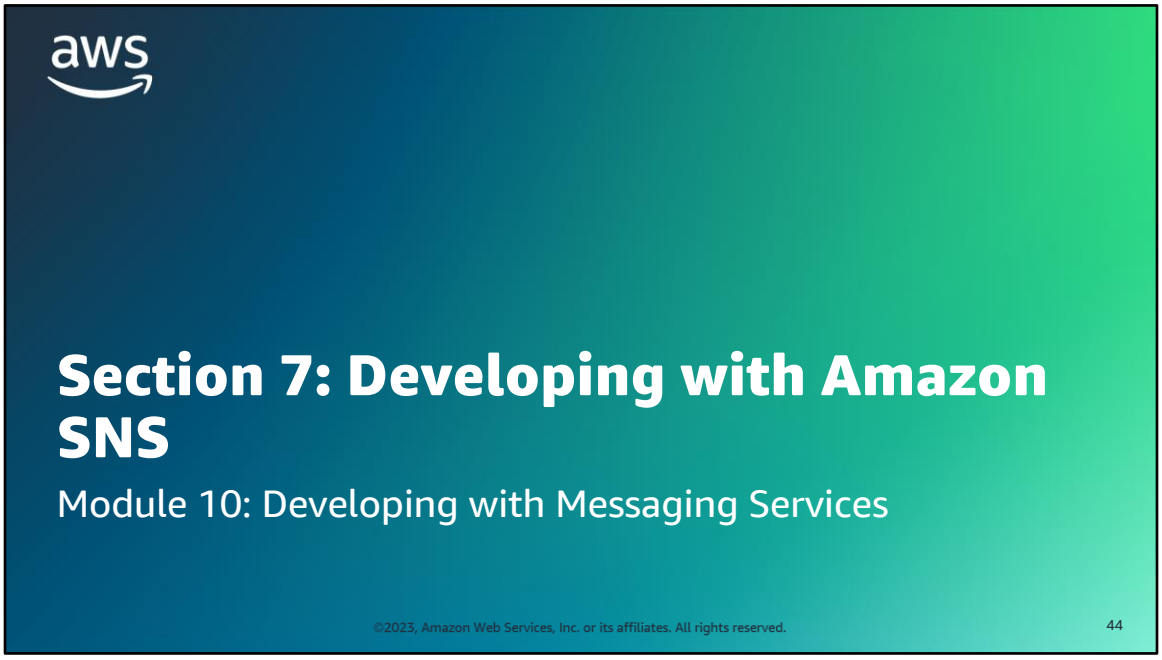
Subscribers in a fanout pattern can be a combination of endpoint types including HTTP endpoints or email addresses.

You could also use fanout to replicate data that is sent to your production environment to your development environment. For example, you could subscribe yet another queue to the same topic for new incoming orders. Then, by attaching this new queue to your development environment, you could continue to improve and test your application by using data received from your production environment. In this example, the DEV queue is connected to a Lambda function that anonymizes data before writing it to a development Amazon DynamoDB database.

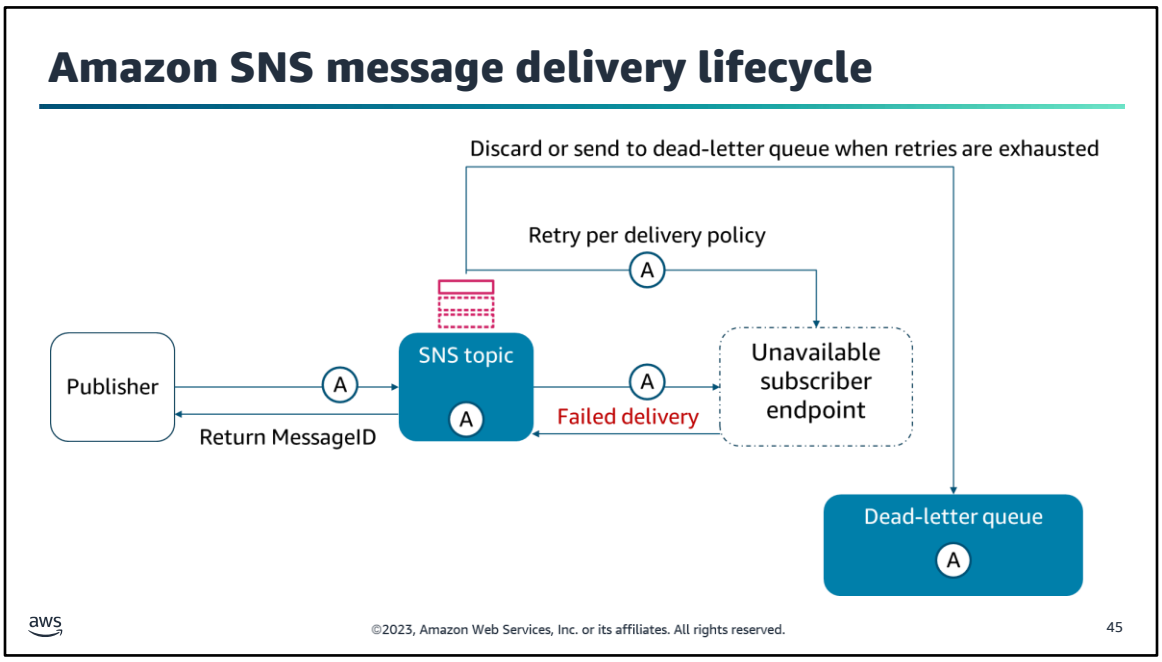


This slide provides an example of how you can use fanout for image processing. When a user uploads images to an S3 bucket for image processing, a message about the event is published to an SNS topic. Multiple SQS queues are subscribed to that topic. When the SQS queues receive the message, they each invoke a Lambda function with the payload of the published message. The Lambda functions process the images (for example, generate thumbnail images, size images for mobile applications, or size images for web applications) and send the results to another S3 bucket.

For more information about fanout and other common Amazon SNS scenarios, see the *Amazon SNS Developer Guide* at https://docs.aws.amazon.com/sns/latest/dg/sns-common-scenarios.html.



Section 7: Developing with Amazon SNS.



When a publisher sends a message to a topic, Amazon SNS returns a message ID and then attempts to deliver the message to all subscriber endpoints.

Amazon SNS defines a delivery policy for each delivery protocol. The delivery policy defines how Amazon SNS retries the delivery of messages when server-side errors occur (when the system that hosts the subscribed endpoint becomes unavailable). When the delivery policy is exhausted, Amazon SNS stops retrying the delivery and discards the message—unless a dead-letter queue is attached to the subscription.

For more information about delivery protocols and policies for each endpoint type, see the *Amazon SNS Developer Guide* at https://docs.aws.amazon.com/sns/latest/dg/sns-message-delivery-retries.html.

Amazon SNS API operations

API operation	Description	Input	Output
`CreateTopic`	Creates a topic	• Topic name	• ARN of topic
`Subscribe`	Prepares to subscribe to an endpoint	• Subscriber's endpoint • Protocol • ARN of topic	• ARN of topic
`ConfirmSubscription`	Verifies an endpoint owner's intent to receive messages	• Token sent to endpoint	
`Publish`	Sends a message to all of a topic's subscribed endpoints	• Message • Message attributes (optional) • Message structure:json (optional) • Subject (optional) • ARN of topic	• Message ID
`DeleteTopic`	Deletes a topic and all of its subscriptions	• ARN of topic	

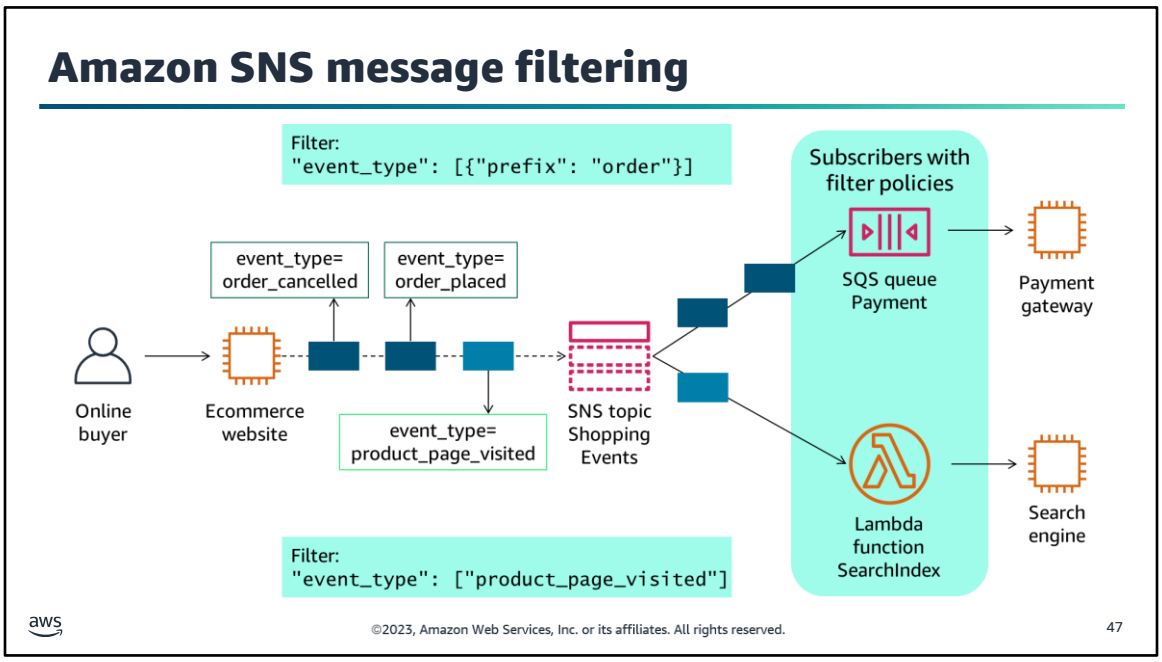
aws

46

Developers should be familiar with the following common API calls for Amazon SNS:

- **CreateTopic:** Creates a topic where notifications can be published. If a requester already owns a topic with the specified name, that topic's Amazon Resource Name (ARN) is returned without creating a new topic.
- **Subscribe:** Prepares to subscribe to an endpoint by sending a confirmation message to the endpoint. If the service was able to create a subscription immediately (without requiring endpoint owner confirmation), the response of the Subscribe request includes the ARN of the subscription. To actually create a subscription, the endpoint owner must call the ConfirmSubscription action with the token from the confirmation message.
- **ConfirmSubscription:** Verifies an endpoint owner's intent to receive messages by validating the token that was sent to the endpoint by an earlier Subscribe action. If the token is valid, the action creates a new subscription and returns its ARN.
- **Publish:** Sends a message to all of a topic's subscribed endpoints. When a message ID is returned, the message has been saved, and Amazon SNS will attempt to deliver it to the topic's subscribers.
- **DeleteTopic:** Deletes a topic and all of its subscriptions. Deleting a topic might prevent some messages that were previously sent to the topic from being delivered to subscribers.

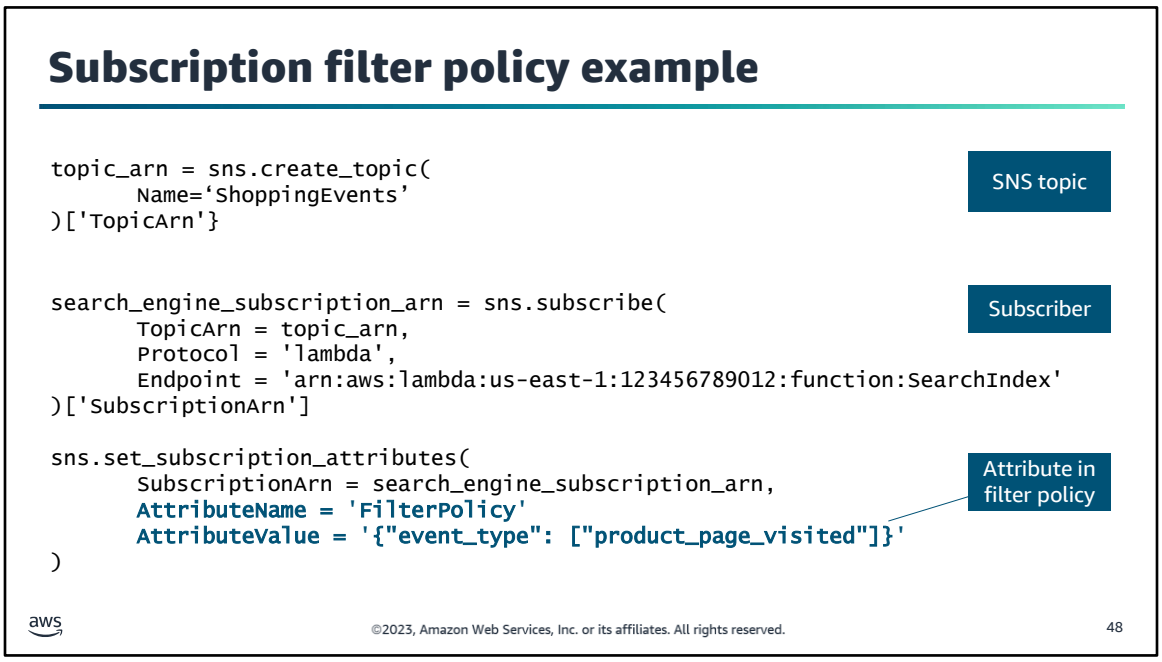
For more information, see the Amazon SNS API Reference at https://docs.aws.amazon.com/sns/latest/api/Welcome.html.



By default, an SNS topic subscriber receives every message that is published to the topic. To receive a subset of the messages, a subscriber must assign a *filter policy* to the topic subscription. A filter policy is a simple JSON object that contains attributes that define which messages the subscriber receives. When you publish a message to a topic, Amazon SNS compares the message attributes to the attributes in the filter policy for each of the topic's subscriptions. If any of the attributes match, Amazon SNS sends the message to the subscriber. Otherwise, Amazon SNS skips the subscriber without sending the message. If a subscription doesn't have a filter policy, the subscription receives every message that is published to its topic.

In this example, an online buyer visits a website, cancels one order, and places another order. These messages are published to the SNS topic *Shopping Events*. The *Payment* SQS queue subscriber has a filter policy applied so it receives only the messages about the orders. The Lambda function subscriber has a filter policy to receive only the messages that the product page was visited.

For more information, see Amazon SNS Message Filtering in the Amazon SNS Developer Guide at https://docs.aws.amazon.com/sns/latest/dg/sns-message-filtering.html.



This slide highlights an example of a topic with a filter policy and example attributes that might be used for filtering messages.

Message with attributes example

```
message = '{"product": {"id": 1251, "status": "in_stock"},'\
          ' "buyer": {"id":4454}}'

sns.publish(
       TopicArn = topic_arn,
       Subject = "Product Visited #1251",
       Message = message,
       MessageAttributes = {
              'event_type': {
                     'DataType': 'String',
                     'StringValue': 'product_page_visited'
              }
       }
}
```

Message

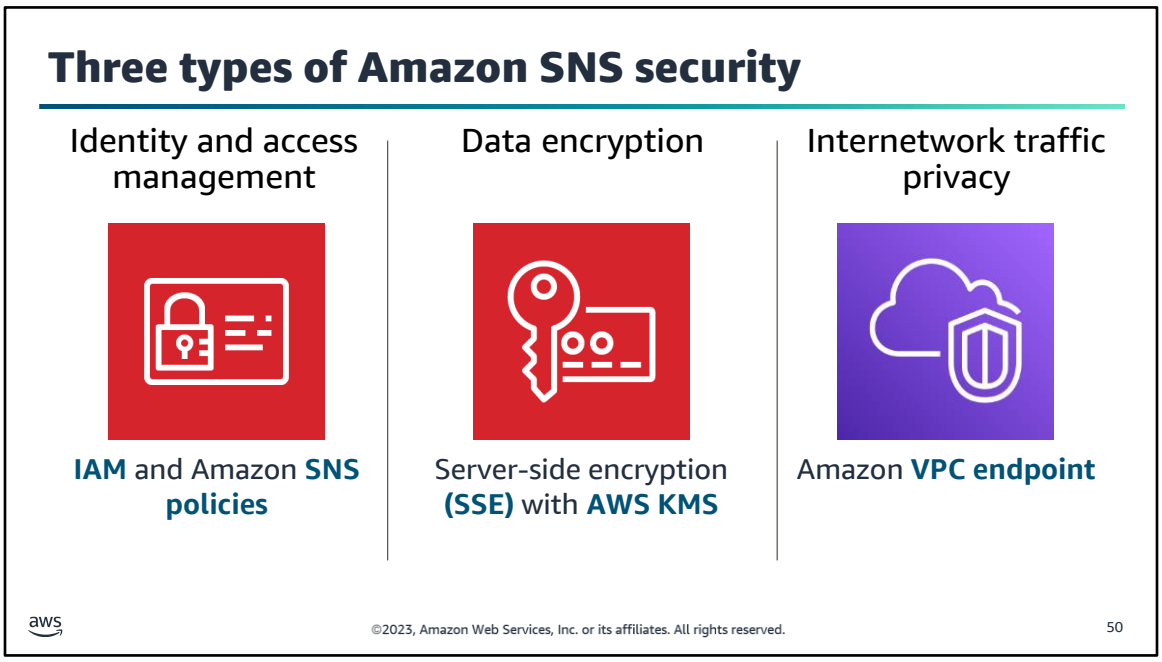
Message attribute that matches the attribute in the filter policy

aws

49

When you publish a message to a topic, Amazon SNS compares the message attributes to the attributes in the filter policy for each of the topic's subscriptions. If any of the attributes match, Amazon SNS sends the message to the subscriber.

In the ecommerce example, the message attribute **event_type** with the value **product_page_visited** matches only the filter policy that is associated with the search engine subscription. Therefore, only the Lambda function that is subscribed to the SNS topic is notified about this navigation event, and the *Payment* SQS queue is not notified.



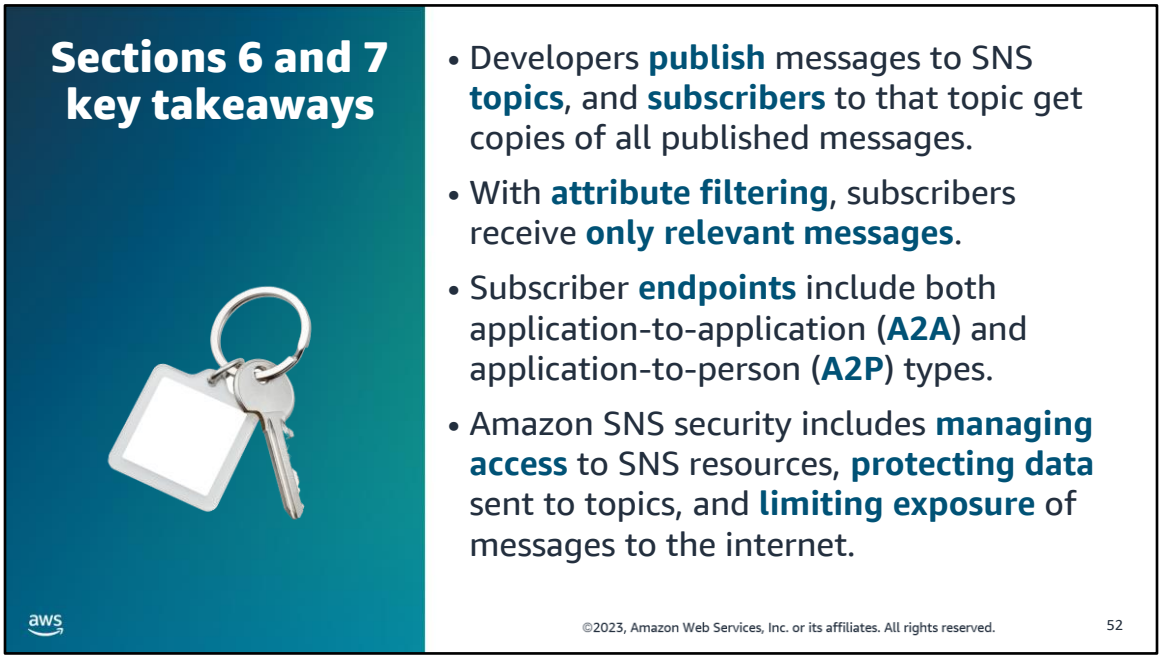
You have detailed control over which endpoints a topic allows, who is able to publish to a topic, and under what conditions:

- **For identity and access management:** Amazon SNS is integrated with IAM so that you can specify which Amazon SNS actions a user in your AWS account can perform with your Amazon SNS resources. You can specify a particular topic in the policy. For example, when you create an IAM policy, you can use variables that give certain users permissions to use the Publish action with specific topics in your AWS account. You can use an *IAM policy* to restrict your users' access to Amazon SNS actions and topics. An IAM policy can restrict access only to users within your AWS account, not to other AWS accounts. You can use an *Amazon SNS policy* with a particular topic to restrict who can work with that topic (for example, who can publish messages to it and who can subscribe to it). Amazon SNS policies can give access to other AWS accounts or to users within your own AWS account.
- **For data encryption:** Encrypt your data using server-side encryption (SSE). SSE protects the contents of messages in SNS topics by using keys that are managed in AWS KMS.
- **For internetwork traffic privacy:** If you use Amazon VPC to host your AWS resources, you can establish a private connection between your VPC and Amazon SNS. With this connection, you can publish messages to your SNS topics without sending them through the public internet.

For more information about Amazon SNS security, see the Amazon SNS Developer Guide at https://docs.aws.amazon.com/sns/latest/dg/sns-security.html.

There is a video demonstration available for this topic. You can find this video within the module 7 section of the course with the title: **Demo Working with Amazon Messaging Services**.

If you are unable to locate this video demonstration please reach out to your educator for assistance.



The following are the key takeaways from this section of the module:

- Developers publish messages to SNS topics, and subscribers to that topic get copies of all published messages.
- With attribute filtering, subscribers receive only relevant messages.
- Subscriber endpoints include both application-to-application (A2A) and application-to-person (A2P) types.
- Amazon SNS security includes managing access to SNS resources, protecting data sent to topics, and limiting exposure of messages to the internet.

Section 8: Introducing Kinesis Data Streams.

Kinesis Data Streams

Amazon Kinesis Data Streams

Fully managed, massively scalable and durable real-time data streaming service that you can use to ingest, buffer, and process streaming data in real time

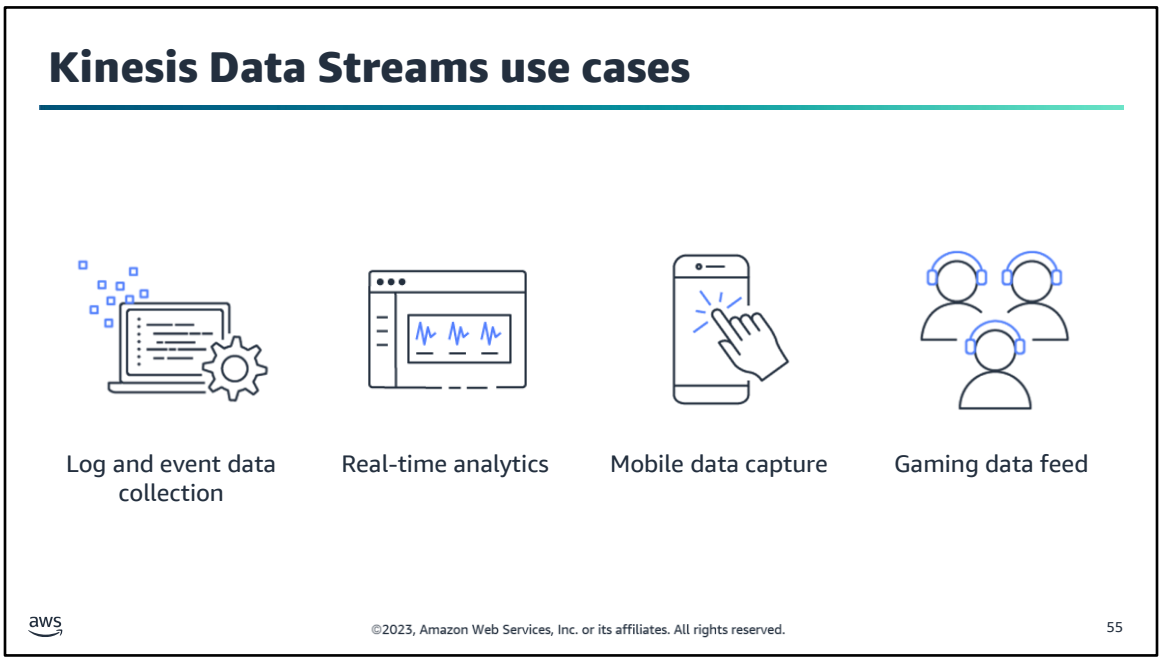
- Continuously capture gigabytes of data per second
- Source stream data from sources including website clickstreams, database event streams, financial transactions, social media feeds, IT logs, and location-tracking events
- Easily process data with built-in integrations with Lambda and other Kinesis services such as Kinesis Data Analytics and Kinesis Data Firehose

aws

54

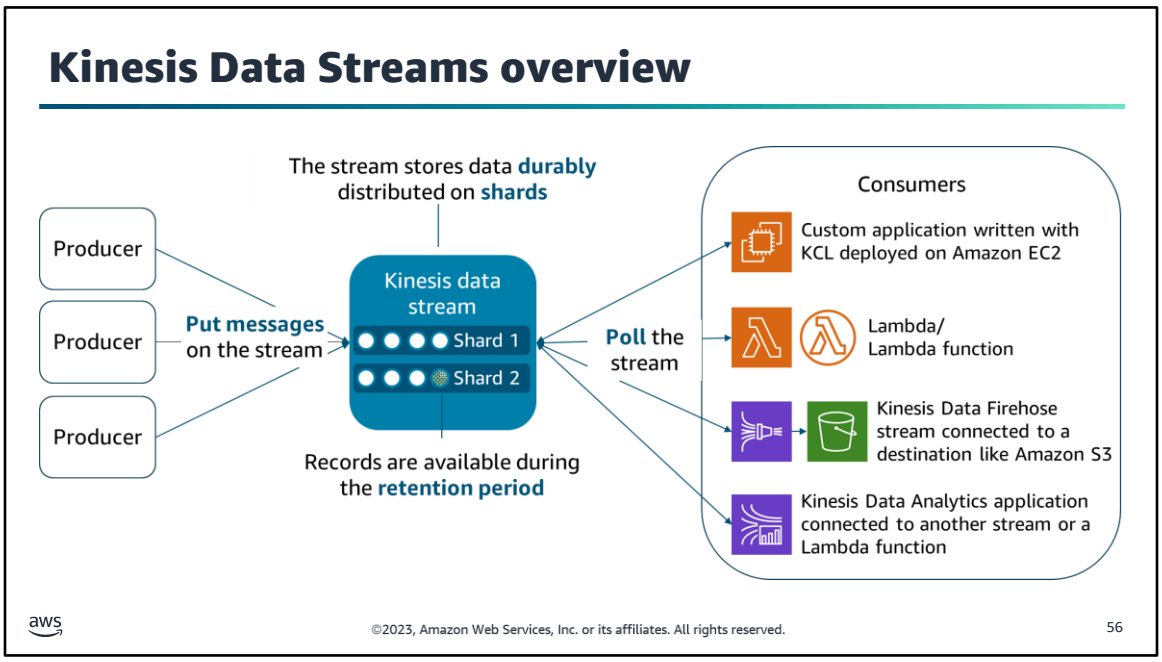
Amazon Kinesis Data Streams is a fully managed data streaming service. Kinesis Data Streams can continuously capture gigabytes of data per second from hundreds of thousands of sources such as website clickstreams, database event streams, financial transactions, social media feeds, IT logs, and location-tracking events. The data collected is available in milliseconds to enable real-time analytics use cases.

Kinesis Data Streams reduces the overhead of building a streaming application with tools including the AWS SDK, the Kinesis Client Library (KCL), connectors, and agents. Kinesis Data Streams also has built-in integrations to AWS Lambda, Amazon Kinesis Data Analytics, Amazon Kinesis Data Firehose, and AWS Glue Schema Registry to simplify setting up a consumer to read records from the stream.



Common Kinesis Data Streams use cases include the following:

- **Log and event data collection:** Collect log and event data, and continuously process the data, generate metrics, power live dashboards, and emit aggregated data into stores like Amazon S3.
- **Real-time analytics:** Run real-time analytics on high-frequency event data such as sensor data.
- **Mobile data capture:** Push mobile application data to a data stream from hundreds of thousands of devices, and make the data available to you as soon as it is produced.
- **Gaming data feed:** Continuously collect data about player-game interactions, and feed the data into your gaming platform.



A producer collects data and puts it on to the stream; for example, a web server that sends log data to a Kinesis data stream is a producer. You can build producers for Kinesis Data Streams using the Kinesis Producer Library (KPL) or the AWS SDK for Java.

Data records also include a partition key and a data blob. The partition key is used to group data by shard within a stream. The sequence number is unique per partition key within its shard. For writes, a shard can support up to 1,000 records per second or up to a maximum of 1 MB per second. For reads, a shard can support up to 5 transactions per second or up to a maximum of 2 MB per second.

Kinesis Data Streams stores a unit of data as a *data record*. A *data stream* represents a group of data records. The data stream ingests a large amount of data in real time, durably stores the data, and makes the data available for consumption. The data records in a data stream are distributed into shards. After a record is added to the stream, the record is available for a specified retention period, which you can set per stream. The Kinesis Data Streams service adds shards to scale horizontally.

Each shard has a uniquely identified sequence of data records, and each data record has a sequence number that Kinesis assigns.

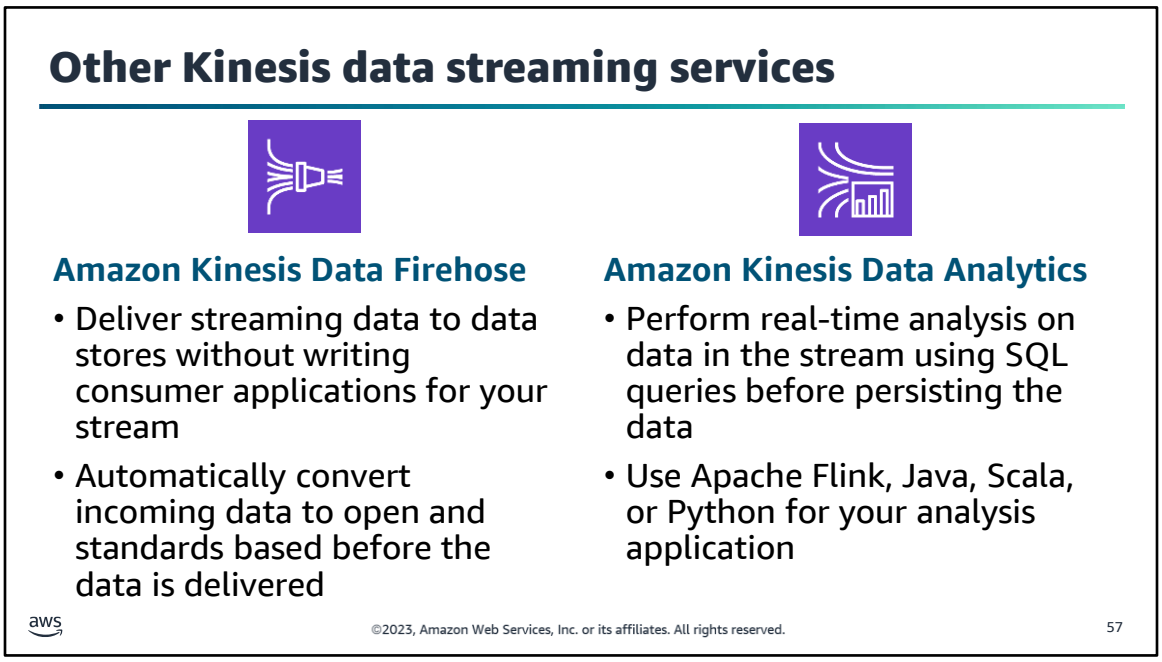
A *partition key* is used to group data by shard within a stream. When you create a stream, you specify the number of shards for the stream. The total capacity of a stream is the sum of the capacities of its shards.

A *consumer* is an application that polls the data stream and processes the data from a Kinesis data stream. You can have multiple consumers on a data stream. When you have multiple consumers, they share the read throughput of the stream among them. An option called *enhanced fanout* lets you give each consumer its own allotment of read throughput.

Create consumers using any of the following options:

- Build your own application to process stream data using the Kinesis Consumer Library (KCL) and deploy it to Amazon EC2.
- Configure a Lambda function as a consumer of the stream to process data serverlessly, and let Lambda handle aspects of consuming the stream.
- Set up Kinesis Data Firehose as a consumer to deliver records to destinations including Amazon S3, Amazon Redshift, Amazon Elasticsearch Service (Amazon ES), and Splunk.
- Send records to a Kinesis Data Analytics application to process and analyze data in a Kinesis data stream using SQL, Java, or Scala.

Note that, unlike queue consumers, stream consumers do not delete records from the stream. Instead, each consumer must maintain a pointer of where they are on the stream.



As noted in the previous slide, consumers of a stream might be other Kinesis streaming services such as Kinesis Data Firehose and Kinesis Data Analytics. These services were created to simplify common Kinesis Data Streams use cases and are often used in conjunction.

Amazon Kinesis Data Firehose is the easiest way to reliably load streaming data into data lakes, data stores, and analytics services. The service can capture, transform, and deliver streaming data to Amazon S3, Amazon Redshift, Amazon ES, generic HTTP endpoints, and service providers such as Datadog, New Relic, MongoDB, and Splunk.

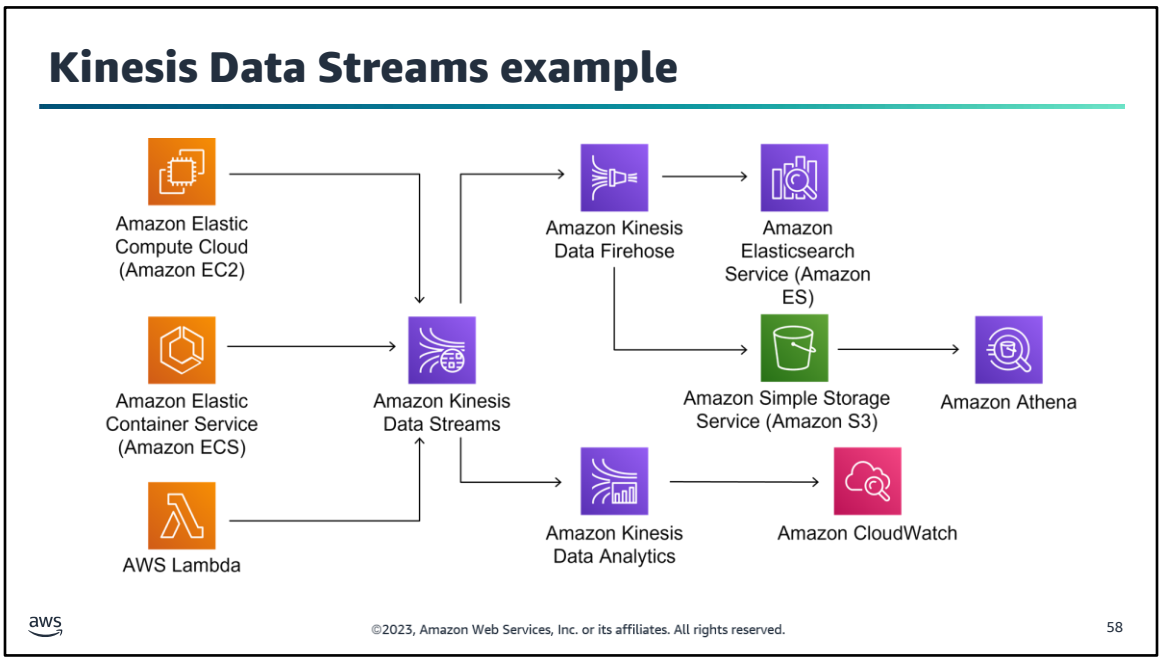
With Kinesis Data Firehose, you can automatically convert the incoming data to open and standards based formats such as Apache Parquet and Apache ORC before the data is delivered.

With Kinesis Data Firehose, you don't need to write consumer applications or manage shards. You configure your data producers to send data to Kinesis Data Firehose, and the service automatically delivers the data to a destination that you specify.

With Kinesis Data Analytics, you can do real-time analysis by using SQL queries before persisting the data. The service is designed for near-real-time queries, and you can aggregate data across a sliding window.

With Kinesis Data Analytics, you write SQL statements or applications, and then upload them into a Kinesis Data Analytics application to perform analysis on the data in the stream. Kinesis Data Analytics also supports using a Lambda function to preprocess the data before your SQL runs.

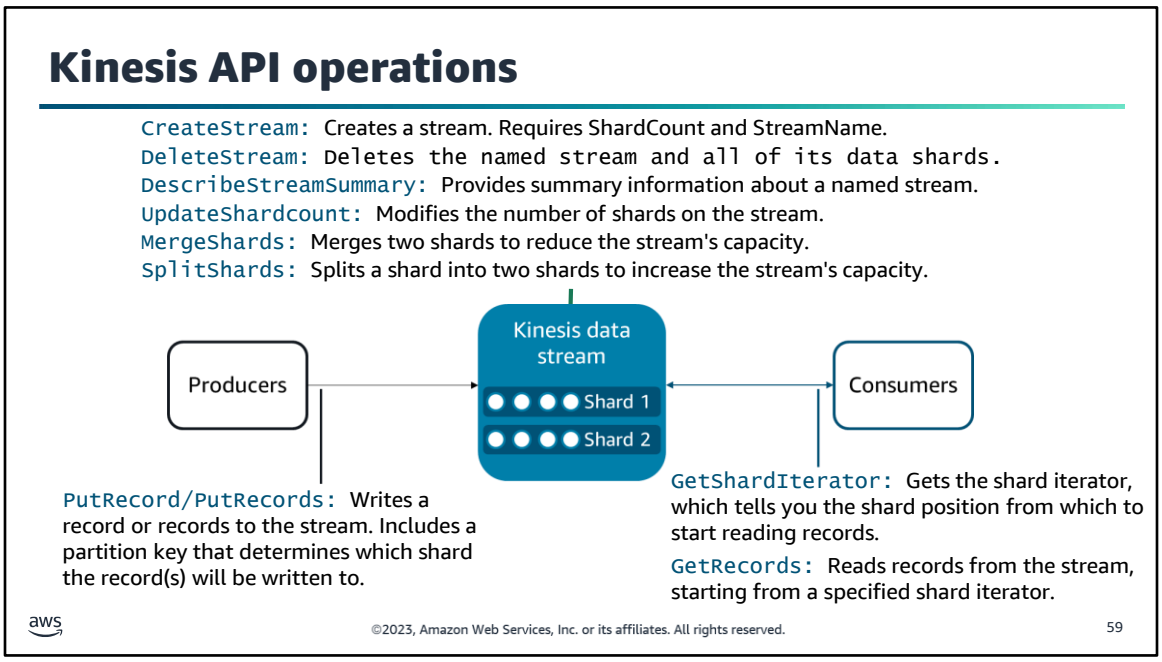
Kinesis Data Analytics applications can enrich data by using reference sources, aggregate data over time, or use machine learning to find data anomalies. Then you can write the analysis results to another Kinesis data stream, a Kinesis Data Firehose delivery stream, or a Lambda function.



In this example architecture, customer interaction logs from multiple applications and microservices running across Amazon EC2, Amazon Elastic Container Service (Amazon ECS), and Lambda are all producers that add records to a Kinesis data stream. Consumers of the stream include Kinesis Data Firehose and Kinesis Data Analytics.

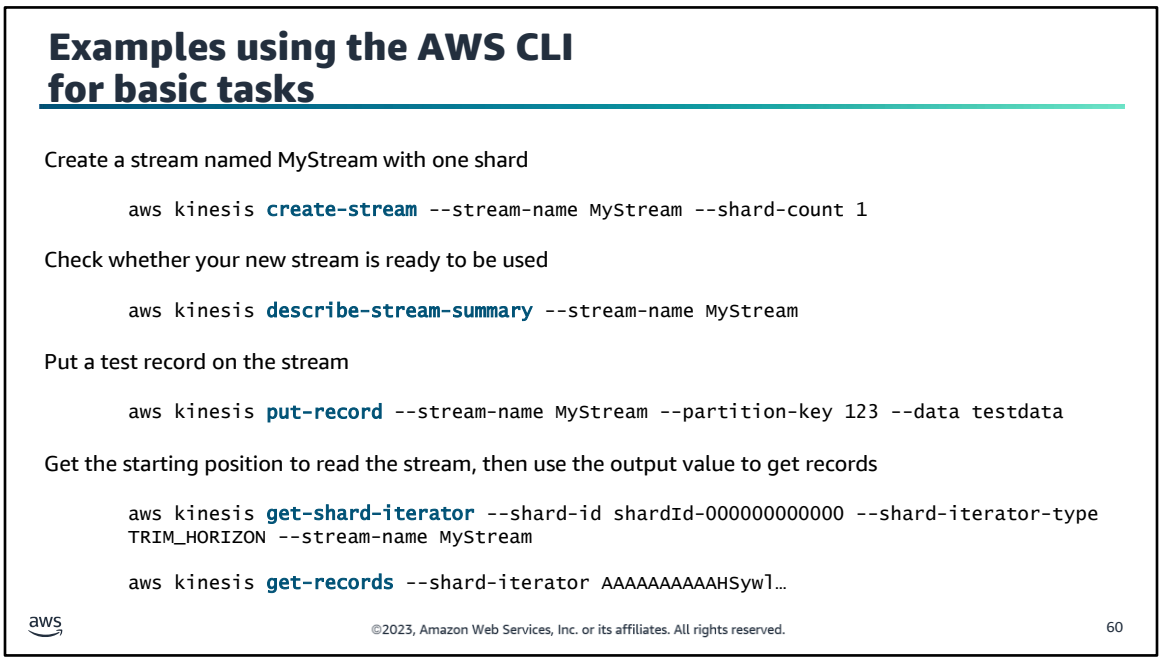
Kinesis Data Firehose delivers the streaming data to two storage destinations: Amazon ES and Amazon S3. The application owners can use Amazon ES for operational insights into their applications. Business users can use Amazon Athena or a similar query tool to interact with the data stored on Amazon S3 without building a complex Extract, Transform, and Load (ETL) processor.

At the same time, Kinesis Data Analytics consumes the stream and generates metrics, percentiles, and derived values. These values are then emitted to Amazon CloudWatch where they are part of an overall monitoring solution.



The Kinesis API provides actions related to managing the stream itself including creating or deleting a stream, or modifying the number of shards on the stream. The API also provides actions that producers use to write records to the stream (PutRecord) and that consumers use to read from the stream (GetRecords).

You might run some of the basic commands from the AWS CLI when learning about streams or when testing. You can manage some stream tasks and AWS integrations from the AWS Management Console. However, you will need other tools to build custom producer or consumer applications. The Kinesis Producer Library (KPL) and Kinesis Client Library (KCL) were built for this purpose.



This slide depicts a few examples of commands that you might use from the AWS CLI to create and do a basic test on a stream.

After you create the stream, you can use the describe-stream-summary command to verify that the stream is in an "active" state before you try to use the stream.

Use the put-record command to write a test record to the stream.

To get records off the stream, first use the get-shard-iterator command to get an identifier for where to start processing stream records. Then, use the get-records command with the shard-iterator value to get records off the stream.

Building producers and consumers

Feature	Kinesis Producer Library (KPL)	Kinesis Client Library (KCL)	Kinesis API with AWS SDK
Abstracts stream management tasks so you can focus on application logic	yes	yes	no
Handles **producer tasks** such as writing with automatic retries and aggregating records to optimize throughput	yes	no	no
Handles **consumer subtasks** such as load balancing and checkpointing processed records	no	yes	no
Provides the ability to write **producer or consumer applications** using direct interaction with the Kinesis API	no	no	yes
Requires your application code to handle stream management	no	no	yes

aws

61

To build your producer and consumer applications, you would typically use the KPL and KCL, or possibly the AWS SDK.

Both the KPL and KCL abstract stream management tasks so that you can focus on application logic. If you need to build custom producer and consumer applications, use the KPL and KCL unless your use case has a specific need that the libraries cannot meet.

The KPL is a highly configurable library that helps you write to a Kinesis data stream. The library acts as an intermediary between your producer application code and the Kinesis Data Streams API actions. The KPL performs the following primary tasks:

- Writes to one or more Kinesis data streams with an automatic and configurable retry mechanism
- Collects records and uses PutRecords to write multiple records to multiple shards per request
- Aggregates user records to increase payload size and improve throughput
- Integrates seamlessly with the KCL to deaggregate batched records on the consumer
- Submits CloudWatch metrics on your behalf to provide visibility into producer performance

The KCL helps you consume and process data from a Kinesis data stream by taking care of many of the complex tasks that are associated with distributed computing. These include load balancing across multiple consumer application instances, responding to consumer application instance failures, checkpointing processed records, and reacting to resharding.

You can build producer and consumer applications by using the AWS SDK to interact directly with the Kinesis Data Streams API. However, for most use cases, use the KPL or KCL to remove the complexities of handling all of the subtasks of stream processing so that you can focus on your application's business logic.

Comparison of queues and streams

	Queues	Streams
Data value	The value comes from processing individual messages.	The value comes from aggregating messages to get actionable data.
Message rate	The message rate is variable.	The message rate is continuous and high volume.
Message processing	Messages are deleted after a consumer successfully processes them.	Messages are available to multiple consumers to process in parallel, and each consumer maintains a pointer but does not delete records.
Example use cases	Financial transactions, product orders	Clickstream data, application logs

aws

62

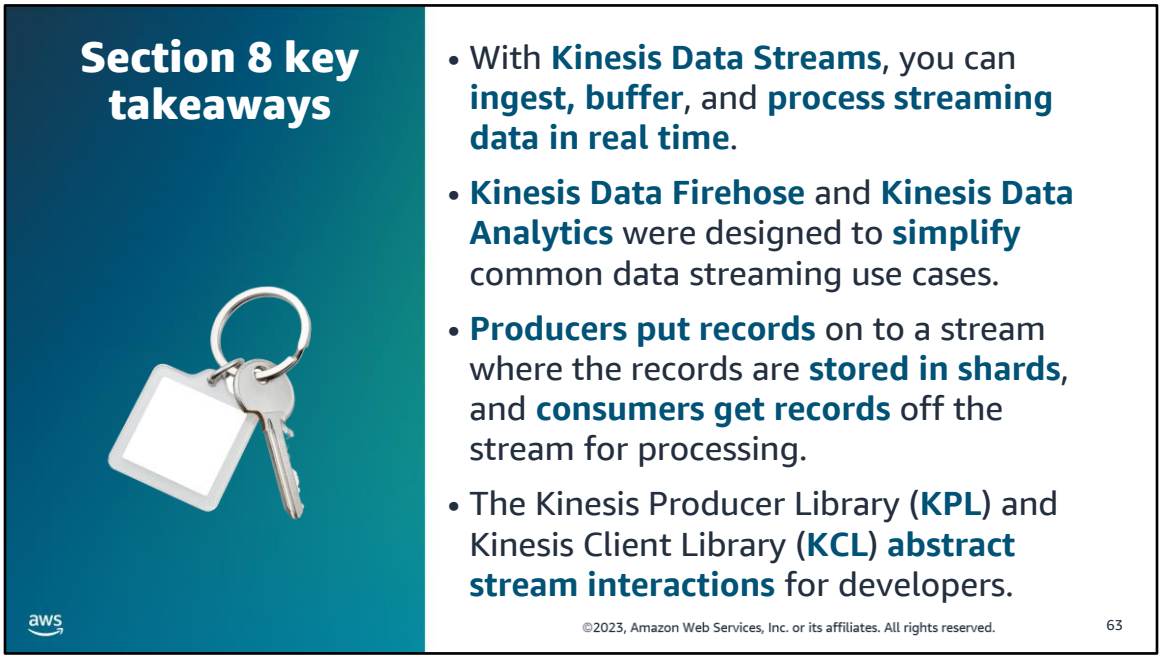
As noted earlier in this module, although both queues and streams are messaging services that use polling to retrieve messages from an interim store, queues and streams are suited to different types of data patterns, and each process the data a bit differently.

The following are characteristics of queues:

- An individual message is the core entity where you apply some processing or compute. The message could be a financial transaction or a command to control an Internet of Things (IoT) device.
- The occurrence of messages typically varies.
- Messages must be deleted off the queue once they have been processed.

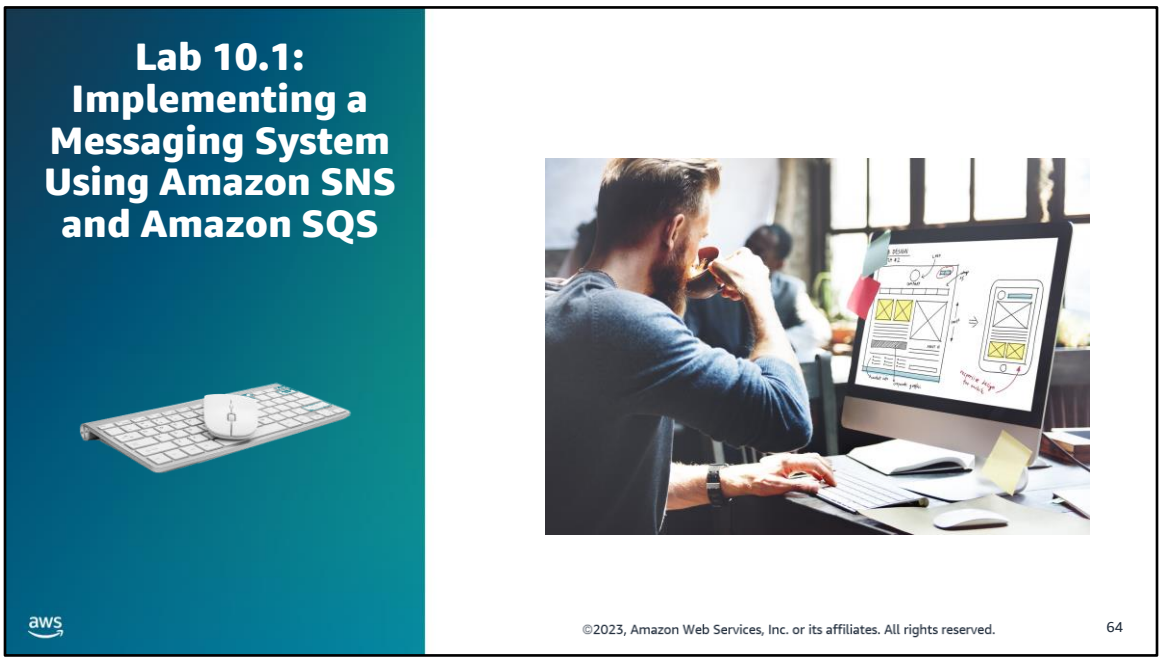
The following are characteristics of stream processing:

- A stream of messages is the core entity. You can only get the value of each individual message by looking at many messages together.
- The stream of messages is also constant in most cases.
- Consumers keep track of where they are on the stream but do not delete records from the stream.
- Clickstream data from mobile apps, application logs, and home security camera feeds are common examples where streams provide the responsiveness that you need to make data actionable.

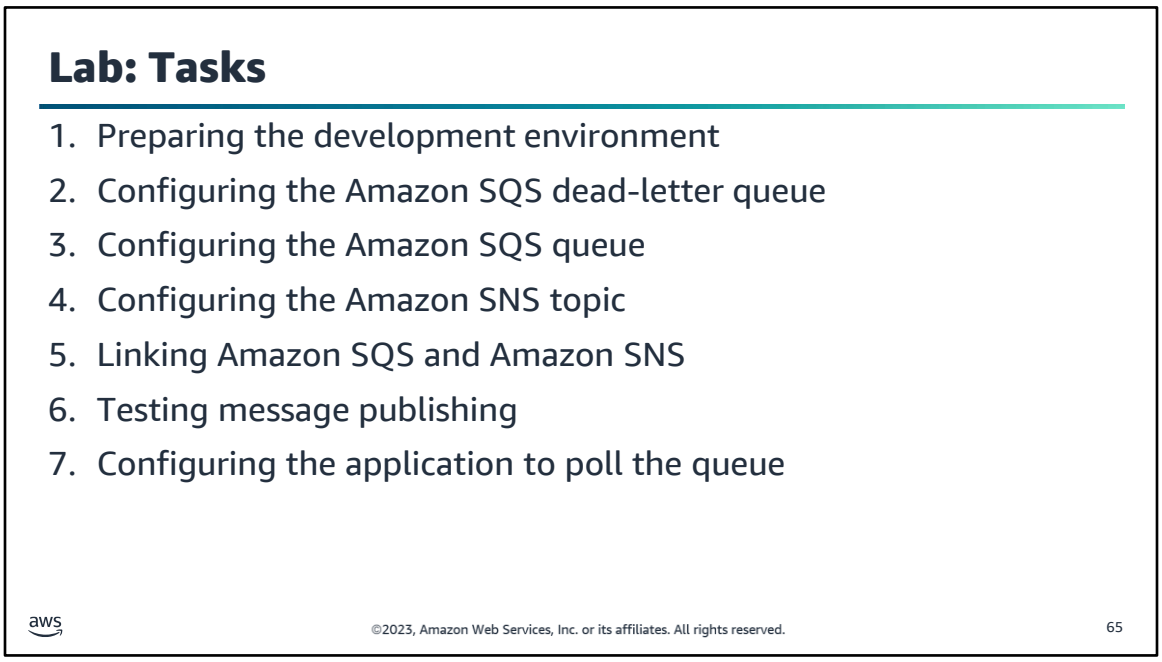


The following are the key takeaways from this section of the module:

- With Kinesis Data Streams, you can ingest, buffer, and process streaming data in real time.
- Kinesis Data Firehose and Kinesis Data Analytics were designed to simplify common data streaming use cases.
- Producers put records on to a stream where the records are stored in shards, and consumers get records off the stream for processing.
- The Kinesis Producer Library (KPL) and Kinesis Client Library (KCL) abstract stream interactions for developers.

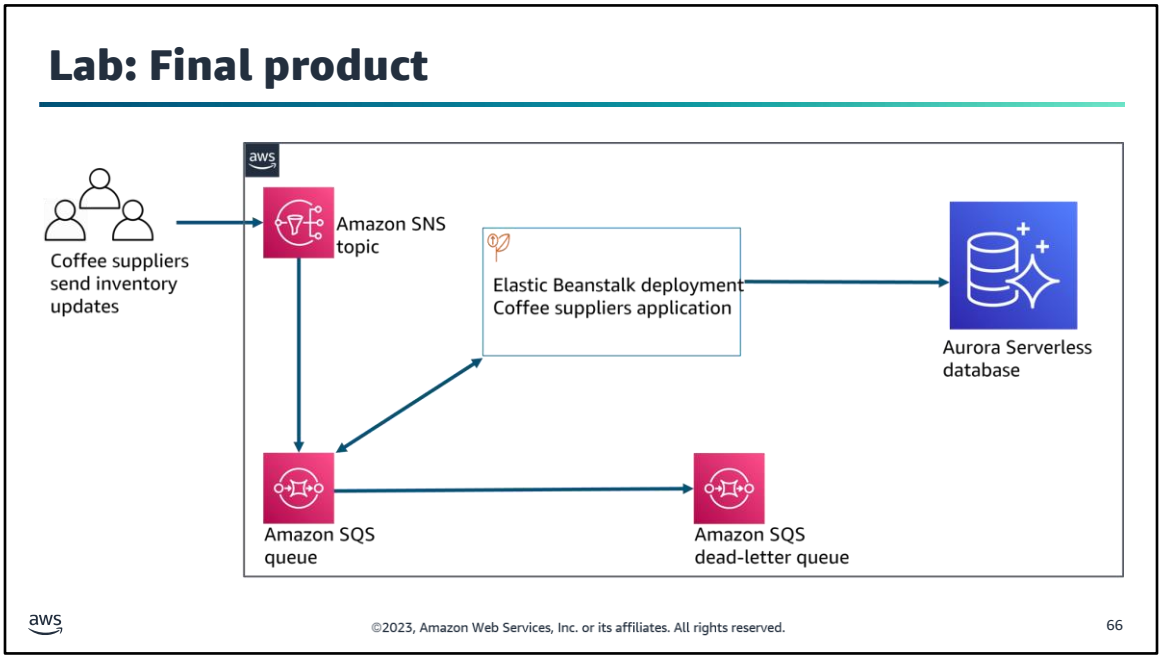


You will now complete Lab 10.1: Implementing a Messaging System Using Amazon SNS and Amazon SQS.



In this lab, you will complete the following tasks:

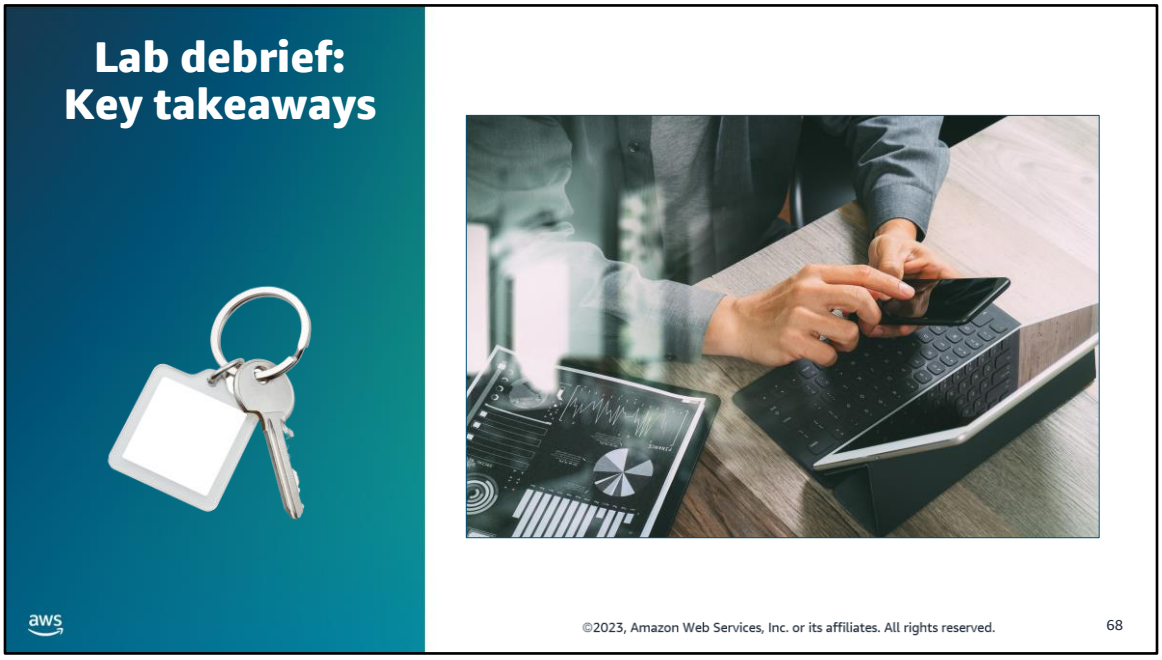
1. Preparing the development environment
2. Configuring the Amazon SQS dead-letter queue
3. Configuring the Amazon SQS queue
4. Configuring the Amazon SNS topic
5. Linking Amazon SQS and Amazon SNS
6. Testing message publishing
7. Configuring the application to poll the queue



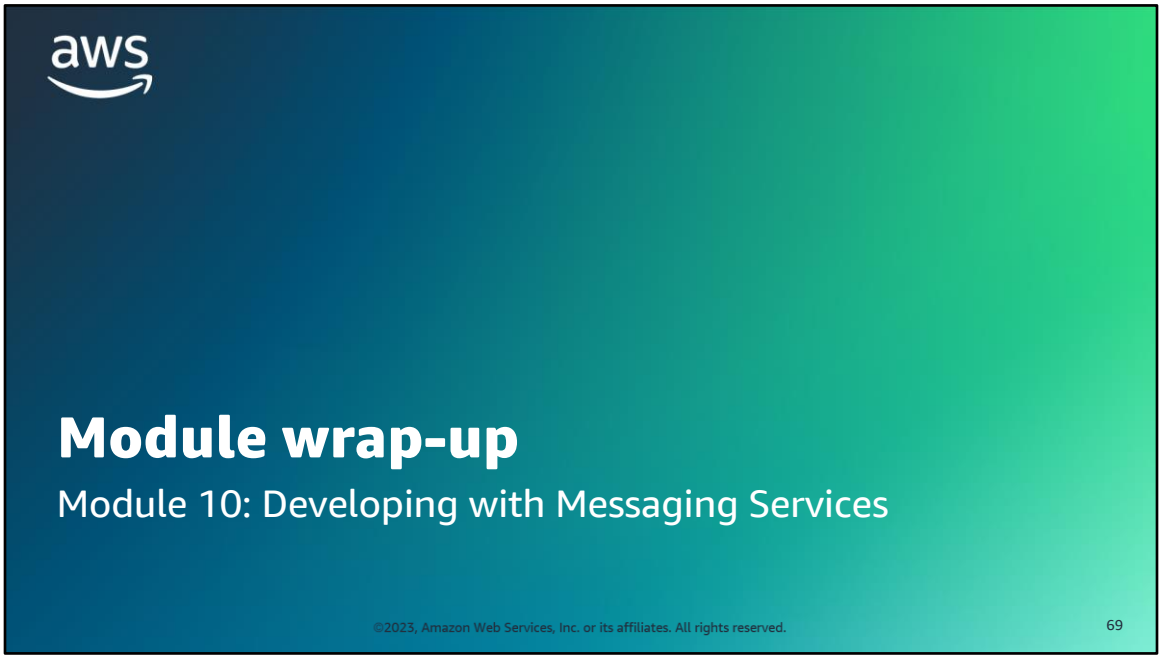
The diagram summarizes what you will have built after you complete the lab.

***For accessibility:** Inventory updates are sent with an SNS topic to an SQS queue. Data then flows to a dead-letter queue or an Aurora Serverless database. **End description.**

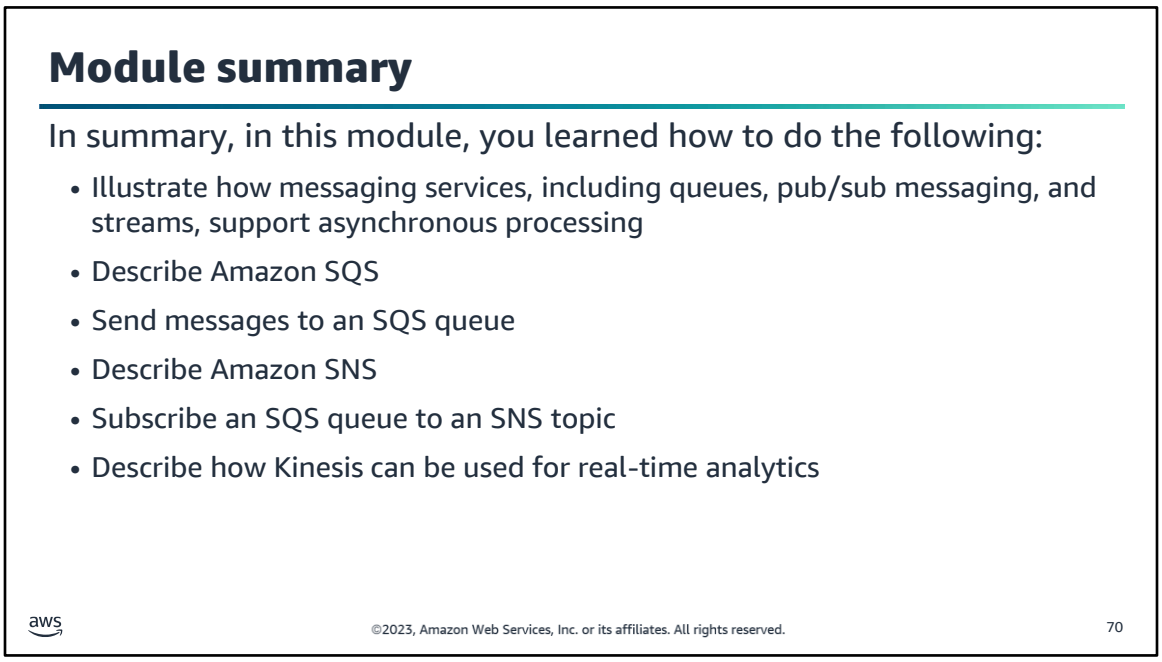
It is now time to start the lab.



After you complete the lab, your educator might choose to lead a conversation about the key takeaways.



It’s now time to review the module and wrap up with a knowledge check, and discussion of a practice certification exam question.



In summary, in this module, you learned how to do the following:

- Illustrate how messaging services, including queues, pub/sub messaging, and streams, support asynchronous processing
- Describe Amazon SQS
- Send messages to an SQS queue
- Describe Amazon SNS
- Subscribe an SQS queue to an SNS topic
- Describe how Kinesis can be used for real-time analytics

Complete the knowledge check

aws

71

It is now time to complete the knowledge check for this module.

Sample exam question

A developer wants to introduce an asynchronous connection in their application workflow that processes individual banking transactions including deposits and withdrawals. Transactions must be handled in the order they arrive. Which method would meet the developer's needs?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- **That processes individual banking transactions**
- **The order**

aws

72

It is important to fully understand the scenario and question being asked before even reading the answer choices. Find the keywords in this scenario and question that will help you find the correct answer.

Sample exam question: Response choices

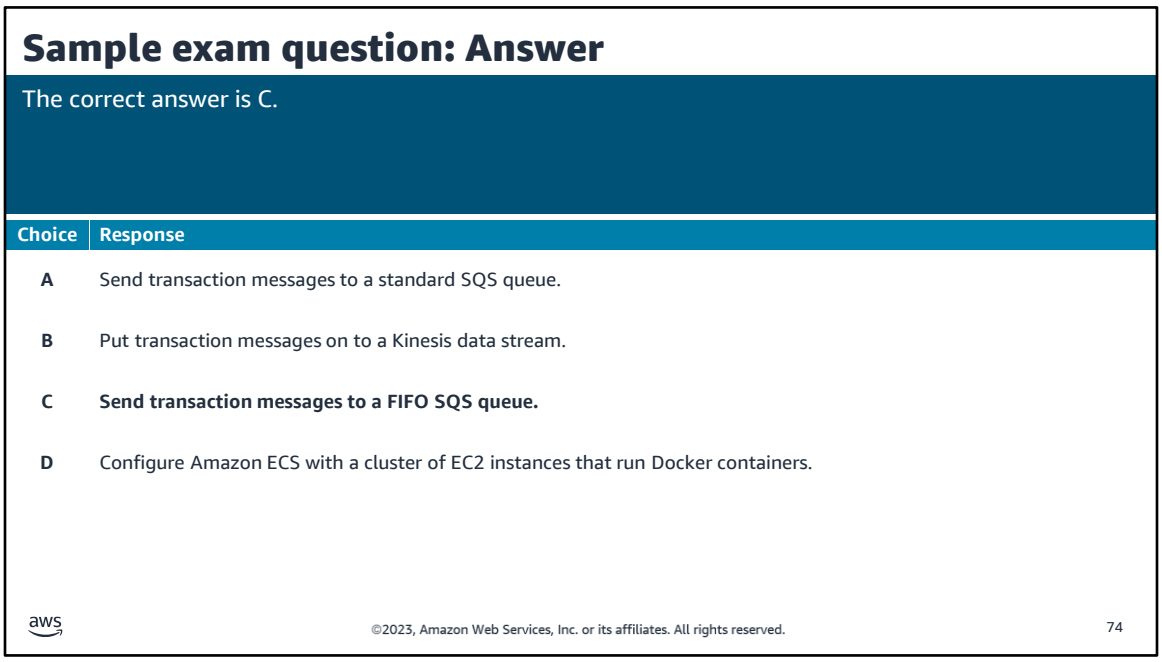
A developer wants to introduce an asynchronous connection in their application workflow **that processes individual banking transactions** including deposits and withdrawals. Transactions must be handled in **the order** they arrive. Which method would meet the developer's needs?

Choice	Response
A	Send transaction messages to a standard SQS queue.
B	Put transaction messages on to a Kinesis data stream.
C	Send transaction messages to a FIFO SQS queue.
D	Configure Amazon ECS with a cluster of EC2 instances that run Docker containers.

aws

73

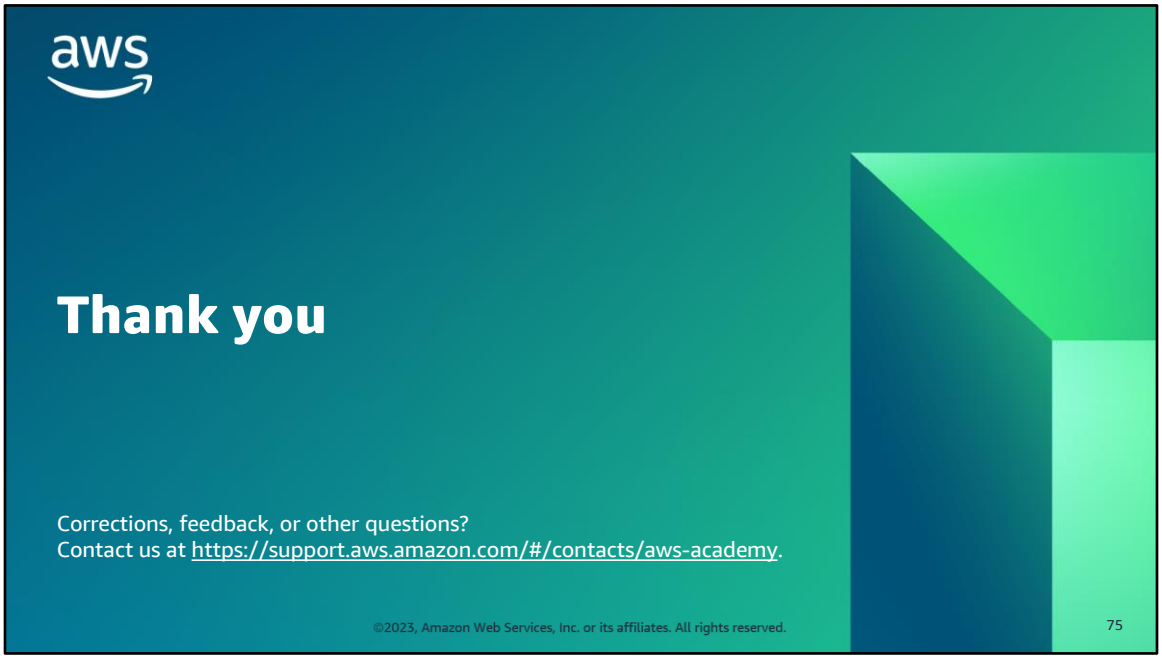
Now that we have bolded the keywords in this scenario, let us look at the answers.



Look at the answer choices and rule them out based on the keywords that were previously highlighted.

The correct answer is **C. Send transaction messages to a FIFO SQS queue**.

Kinesis Data Streams does maintain record order when processing records on a stream, but the description of individual transaction processing implies that you need a queue. To ensure in-order processing, you need a FIFO queue instead of a standard queue.



Thank you for completing this module.